



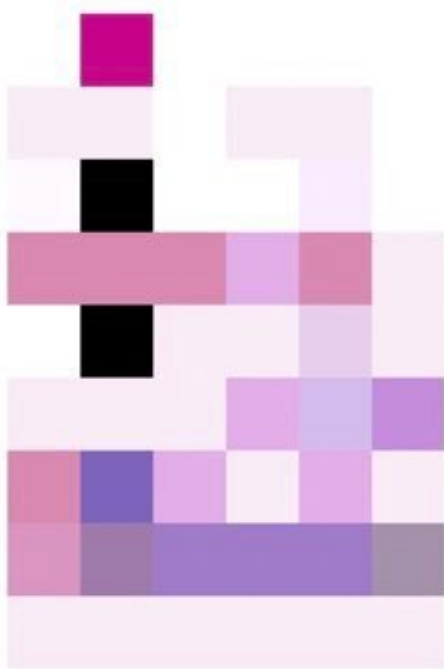
3>SHAKE

意志を実装する アーキテクチャ モダナイゼーション

レガシーを競争優位性へ転換する
ソシオテクニカルなアプローチ

2026/02/19 Developers Summit 2026

@nwiizo 40min (12:40~13:20)



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。格闘技、読書、グラビアが趣味でよく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていく。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



スリーク

日本のSREをリードする

SRE総合支援からセキュリティ対策を全方位支援

Data Engineering



レコナー

**あらゆるサービスを
連携するハブになる**

クラウド型ETL/データパイプ
ラインサービスの決定版

Security



セキュリファイ

**事業者が抱える
セキュリティリスクを無くす**

ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



リランス

**良いエンジニアに
良い案件を**

フリーランスエンジニアに
「今よりいい条件」を



Sreakeのお仕事

クラウドネイティブなアプローチで、お客様の事業をより安全に、競争力のあるサービスへ

提供サービス

SRE/DevOps支援

- Kubernetes構築・運用
- クラウドネイティブ化推進
- Observability導入

アーキテクチャモダライゼーション

- 現状分析・戦略策定
- 段階的な移行支援
- 内製化・伴走支援

データ活用支援

- データ基盤構築
- BigQuery/Snowflake
- 分析基盤最適化

ご依頼・ご相談お待ちしております

<https://sreake.com/>



We are Hiring!!

3-shakeと一緒にSRE界隈を盛り上げてくれる**仲間を大募集中**です！

Mobility、FinTech、通信など大規模SREを存分に経験できます

是非、カジュアル面談しましょう！

リードSREエンジニア
SREを突き詰めたい方



SREエンジニア
SREを始めてみたい方



フルスタックエンジニア
モダンスタックでSREと開発も両方
やりたい欲張りな方



本日のアジェンダ

1. AI時代に私たちの仕事はどう変わるのか
2. ソフトウェアと組織を同時に設計する
3. ビジネスを可視化する協働的手法
4. ドメイン駆動設計とチーム設計の統合
5. レガシーを競争優位性へ転換する
6. 意志を実装するということ



この発表で解決できること

こんな悩みを持っていませんか？

レガシーシステムの改善が進まない。技術的な改善と組織の問題が絡み合っている。AIでコードは書けるが、**設計の判断ができない**。

この発表で持ち帰れるもの

アーキテクチャモダライゼーションの全体像、ソシオテクニカルな視点での設計アプローチ、そして**協働的な設計手法**（イベントストーリーミング等）。

目標：「どんな未来を描くか」を考えるための武器を持ち帰る



この発表の限界

本日紹介する考え方は、すべての現場に当てはまるわけではありません。

文脈によって最適解は変わる

スタートアップかエンタープライズか、B2CかB2Bか、規制産業かどうか——これらによって正解は異なる。「うちには当てはまらない」と思う部分がある。それは正しい反応。

想定される反論に先に答えておく

「そんな余裕はない」→ 余裕がないからこそ構造を見直す価値がある。「うちは小さいから関係ない」→ 小さいうちに考えておく方が楽。「理想論だ」→ 理想を知った上で、現実とすり合わせるのが意志の実装。

正解は存在しない。だが「考え方の型」は存在する。それを持ち帰ってほしい



始める前に

**事情でレガシーシステムと呼ばれ
敵にされる全てのシステムに敬意を表します。**

それらのシステムは、かつて誰かが全力で作ри、ビジネスを支え、価値を生み出してきた。

敬意の次に必要なのは責任。 今日書くコードも、いつかレガシーになる。その時に「あの時代の制約の中で、よくここまで作った」と言われるか、「なぜこんな設計にしたのか」と言われるか。それを決めるのは今日の選択。



1. AI時代に私たちの仕事は どう変わるのか

AIがコードを生成する時代

AIにできること

ボイラープレートの生成、パターンに従った実装、既知の問題の解決。**再現可能な作業**はAIが得意とする領域。

AIにできないこと（まだ）

「何を作るべきか」の判断、ビジネス固有の文脈理解、組織や人の問題への対処。**正解のない問い**には人間が必要。

Claude Codeの登場から1年。コード生成の精度は上がり続けており、この流れは不可逆で加速する一方だ。

実装力はコモディティ化した。差がつくのは「何をどう作るか」の判断力と、組織を動かす合意形成力になってきた。



なぜ「決める」ことが難しいのか

「人間が決めるべき」は正論。だが現場では決められていない。

組織構造の問題

プロダクトオーナーは「要件をまとめる人」、エンジニアは「言われたことを作る人」。「なぜ作るか」を問う役割が不在。

心理的な問題

決めると責任が発生する。「正解がわからない」から決められない。**上位者の承認を待つ癖**がついている。

情報の問題

ビジネス側と技術側で情報が分断。「何が可能か」と「何が必要か」が噛み合わない。**判断材料が揃う頃には手遅れ**。

AIがコードを書いても、この構造的問題は解決しない。むしろ露わになる。



AIは責任を取ることができない（まだ）

AIは問いを立てることもできる。答えを出すこともできる。

でも、その結果に責任を取ることができない。少なくとも今は。

AIができること

選択肢を提示し、トレードオフを分析し、過去の事例を参照する。**判断材料を揃えることはできる。**この能力は日々向上している。

人間にしかできないこと（まだ）

「これでいく」と決め、結果を引き受け、ステークホルダーに説明する。**最終的な責任を負うことは今のところ人間だけの仕事。**

AIは答えを出せる。だが責任は取れない——まだ。



AIエージェントという新しい存在

AIは「対話相手」から「自律的な実行者」へ変わりつつある

コード生成、テスト実行、デプロイ、障害対応——AIエージェントが自律的にタスクを遂行する時代が来ている。ただ、これは新しい話ではない。CI/CDパイプラインも、自動スケーリングも、広い意味では「自律的な実行者」だった。

AIエージェントが変えること

タスクの粒度が変わる。「関数を書いて」から「この機能を実装して」へ。**委任の単位が大きくなる**ほど、委任する側に求められる判断力も高くなる。

AIエージェントが変えないこと

「何を作るべきか」の判断、組織間の合意形成、ビジネスの文脈理解、そして**失敗したときに顧客の前に立つ覚悟**。これらは依然として人間の領域。

エージェントが賢くなるほど、委任する側の判断力が問われる



AIエージェントとの信頼関係を設計する

AIエージェントは「わかったふりをする優秀な新人」に似ている

新人に「全権委任」はしない。小さなタスクから任せ、成果を確認し、徐々に委任範囲を広げる。AIエージェントも同じ。哲学者キャサリン・ホーリーは信頼を「**相手がコミットメントを果たすとあてにすること**」と定義した。AIには「意図」がないが「能力」は日々向上している。だから**コミットメントを人間が定義し、AIが履行する**構造を作る。

すぐに委ねてよいもの

定型的なコード生成、テストの実行と結果の解析、ドキュメントの下書き、既知パターンの適用。**正解が検証可能な領域**。

いきなり委ねてはいけないもの

ドメイン境界の決定、チーム構造の設計、技術的負債の許容判断、ステークホルダーへの説明。**正解が文脈に依存する領域**。ただし「永遠に委ねられない」ではない。

CLAUDE.mdやrulesファイルは「暗黙知の形式知化」だ。人間同士なら空気で伝わるものを、AIに伝わる言葉に翻訳する作業。ホーリーは「不信に値する相手には不信を」とも説く。**適切な不信の設計**——レビュー、テスト、段階的な権限委譲——こそが鍵になる。

信頼とは感情ではなく設計。コミットメントを明文化し、履行を検証する仕組みを作れ。



Harness Engineering

「適切な不信の設計」を技術的に実装するとどうなるか。Birgitta Böckeler (Thoughtworks) は、AIエージェントが大規模コードベースを保守するために必要なツールと慣行の総体を **Harness Engineering** と呼ぶ。

Context Engineering

CLAUDE.mdやrulesのようなコードベース内の知識ベースを継続的に拡充し、AIエージェントに正しい文脈を与える。前スライドで言う「コミットメントの明文化」そのもの。

Architectural Constraints

カスタムリンター、構造テスト、ArchUnitなどの**決定論的な制約**でAIの出力を監視する。LLMの判断だけに頼らない。

Garbage Collection

ドキュメントの矛盾やアーキテクチャ制約違反を定期的に検出するエージェント。放置すればエントロピーは増大する。

AIに自由にコードを書かせるな。制約という「馬具 (harness)」で方向を定めよ。



AIが変えるもの、変えないもの

信頼の構造と制御の技術がわかったところで、一歩引いて考えたい。AIの進化で、本当に変わるものは何か。

変わるもの

実装の速度、コード生成のコスト、情報収集の効率、プロトタイピングの手軽さ。要するに「**How**」のコストが劇的に下がる。

変わらないもの

コンウェイの法則、認知負荷の限界、組織の慣性、人間の感情。そして信頼の構築には時間がかかるという事実。コミットメントの積み重ねに近道はない。要するに「**人間の性質**」は変わらない。

だからこそ、ソシオテクニカルなアプローチの価値は**AIが発達するほど高まる**。技術側のコストが下がれば下がるほど、ボトルネックは「人間の調整コスト」に移動する。AIエージェントが100倍速くコードを書いても、チーム間の合意形成が3ヶ月かかるなら、全体のリードタイムは3ヶ月のまま。信頼の醸成も、組織の慣性の突破も、AIには代替できない。

AIは「How」を加速する。だが「What」と「Why」は加速しない。ここが今日の出発点。



歴史は繰り返す——「作れる」と「運用できる」

新しい技術が出るたびに、同じサイクルが回る

「魔法のような技術」に触れる → 「これなら自分たちで作れる」 → 内製する → 運用が回らなくなる → 結局外部に移行する。クラウド黎明期にもコンテナブームでも、そして今のAIでも、同じことが起きている。

技術の進歩は「作る」コストを劇的に下げる。AIが書いたコードは動く。だが、そのコードを3年間運用し、法改正に追従し、セキュリティパッチを当て続け、担当者が退職しても回し続けるコストは下がらない。歴史的教訓を持たない世代が新技術に触れるたびに同じ興奮を覚え、「作れた」の興奮と「運用し続けられる」の地味な現実との落差に沈む。組織・法制度・業務フローは常に変化するため、システムは「完成」しない。作った瞬間から陳腐化が始まる。

「作れること」と「運用できること」の間には、技術では埋まらない溝がある

エンジニアの仕事の質的シフト

AIが「作る」コストを下げ、歴史が「運用は別物」だと教えている。ならばエンジニアの仕事はどこへ向かうのか。

以前

コードを書き、技術を選び、機能を実装し、バグを直す。**実装そのものが仕事**だった。

これから――意志を実装する

どんな価値を届けるか（**ビジネスへの意志**）、どこに境界を引くか（**アーキテクチャへの意志**）、チームをどう分けるか（**組織への意志**）。技術の選択、設計、プロダクトがもたらす価値への責任を負うこと。

「作れる人」は余る。「何を作り、何を作らないか決められる人」が足りなくなる



技術が楽になると、本題が見える

開発コストの低下がもたらす逆説

コーディングが楽になる → 開発のボトルネックが移動する → 「人間の問題」が前面に出てくる。制約理論の考え方だ。ボトルネックを1つ解消すると、次のボトルネックが見えてくる。

以前は隠れていた

「技術的負債のせいで遅い」「レガシーが複雑すぎる」
「リソースが足りない」——**技術的な言い訳**ができた。

今は露わになる

「何を作るか合意できない」「チーム間の調整が進まない」
「誰も決められない」——**人間の問題**が見える。

これらは「面倒な雑務」ではない。合意形成、責任境界の設計、組織文化の変革、知識の共有と継承——**ソフトウェア開発の本質的な難しさ**であり、AIが発達してもなくなるらない。

コードが楽に書ける時代、言い訳はもう使えない。これは「面倒」ではなく「本題」。



では、どう向き合えばいいのか？

「人間の問題に向き合う」とは、具体的に何をするのか？

抽象的な理解だけでは動けない。私たちには実践的なフレームワークと手法が必要。

必要なもの

- ソフトウェアと組織を同時に設計する方法
- ビジネスを協働で可視化する手法
- チームの自律性を確保するパターン

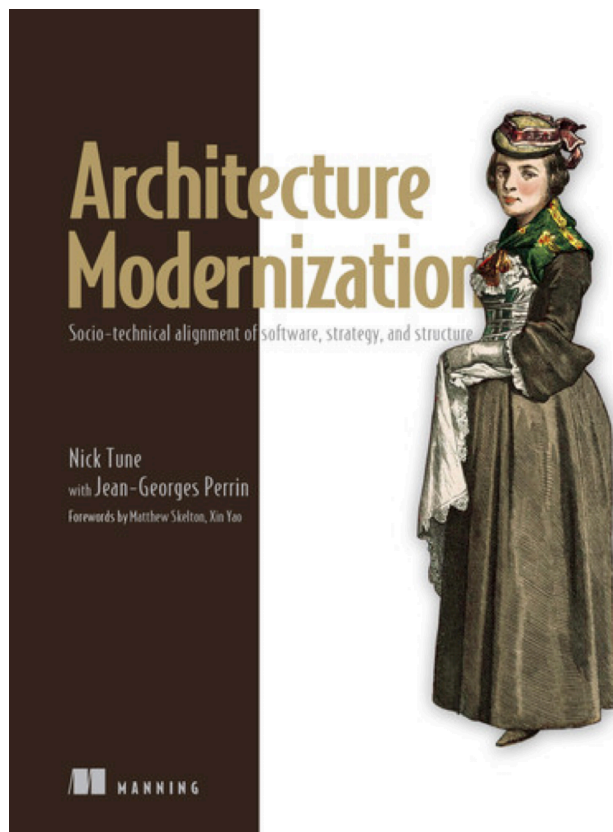
これから紹介する内容

- ソシオテクニカルアーキテクチャ
- イベントストーリーミング
- DDD × Team Topologies

私がこれから紹介するのは、その「面倒な本題」に向き合うための道具。



「Architecture Modernization」という書籍



書籍の概要

- 著者: Nick Tune, Jean-Georges Perrin
- レガシーシステムを競争優位性に転換する包括的ガイド
- ソシオテクニカルな視点を中心に据えた実践書

本書のアプローチ

1. モダナイゼーションの「Why」を理解する
2. モダンアーキテクチャを「Design」する
3. 変革を「Execute」する

日本語版が出ました

アーキテクチャ モダナイゼーション

組織とビジネスの未来を設計する

Nick Tune, Jean-Georges Perrin 著

株式会社スリーシェイク

元内 柊也／岩崎 勇生／角谷 太雅／加藤 岳明／佐藤 慧太 訳

Architecture Modernization
Socio-technical alignment of software, strategy, and structure

何度書き直しても、
また遅くなる。

- ▶ その徒労感の正体は、コードではなく組織構造にある。
- ▶ 組織の形がシステムの形を決め
コンウェイの法則から逃れられる組織はない。
- ▶ DDD、チームトポロジー、ワードリーマッピング
——世界の現場が採用した処方箋。

SE
SHAKE



アーキテクチャモダナイゼーション

「何度書き直しても、また遅くなる。」 — 本書冒頭より

- 翻訳: 株式会社スリーシェイク（5名で翻訳しました）

翻訳作業は、扱う用語の範囲が広く、適切な日本語がまだ定まっていない用語も多くて大変でしたが、やりがいのある仕事でした。

Developer Summit 2026でサイン会があります！

書籍の構成（17章）

Part 1: Why（理解）

- Ch.1 アーキテクチャモダライゼーションとは
- Ch.2 旅への準備
- Ch.3 ビジネス目標

Part 2: Discovery（発見）

- Ch.4 リスニング&マッピングツアー
- Ch.5 Wardley Mapping
- Ch.6 プロダクトタクソノミー

Part 3: Design（設計）

- Ch.7 イベントストーリーミング
- Ch.8-9 ドメイン境界の特定

Part 4: Organization（組織）

- Ch.10 戦略的ITポートフォリオ
- Ch.11 Team Topologies

Part 5: Technical（技術）

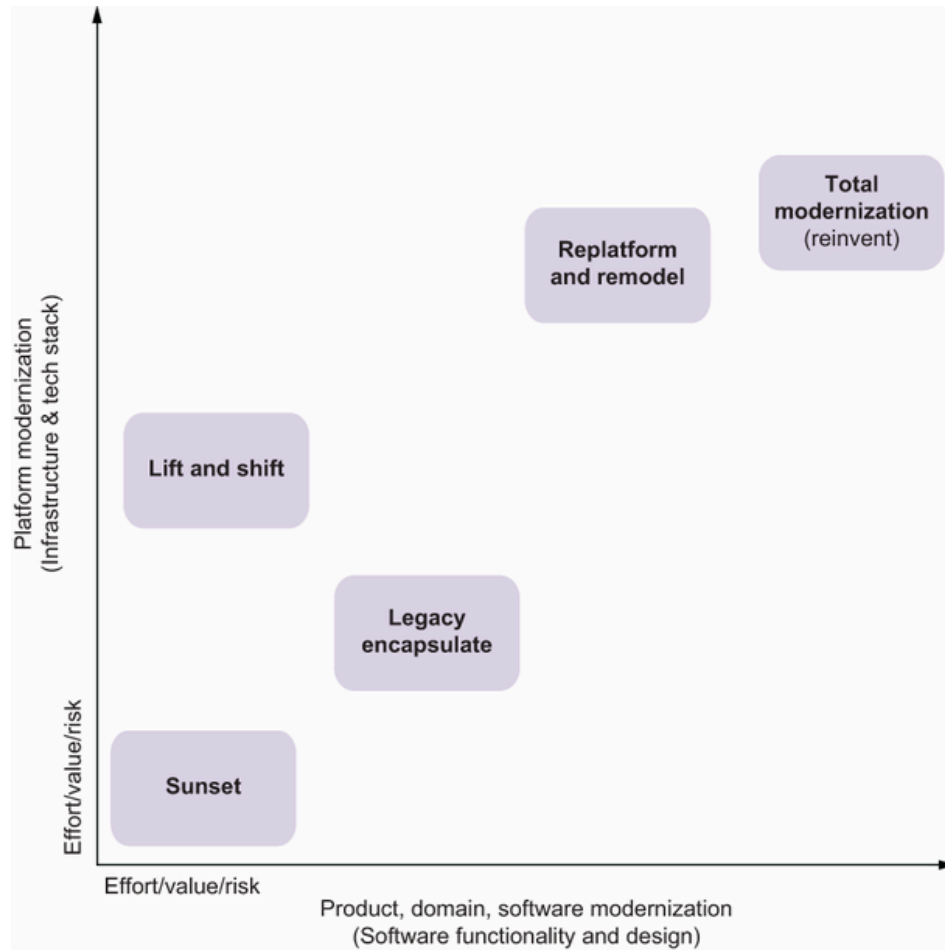
- Ch.12-14 疎結合アーキテクチャ、IDP、Data Mesh

Part 6: Execute（実行）

- Ch.15-17 AMET、戦略とロードマップ、学習



モダナイゼーション戦略セレクター



「全部作り直す」も「全部残す」も間違い。サブドメインごとにビジネス価値と複雑性を測り、**どこに投資すれば最もリターンが大きい**かを判断する。

この図が示すのは、モダナイゼーションとは技術の問題ではなく**投資判断の問題**だということ。限られたリソースをどこに集中させるか——その意思決定のフレームワークがStrategy Selectorである。

Figure 1.10 Identifying the optimal modernization return on investment per subdomain with the Modernization Strategy Selector より引用



現代システムの「三体問題」

「技術」「組織」「戦略」の3つが相互に影響し合う

3つの要素が互いに引力を及ぼし合い、予測不可能な振る舞いをする。どれか1つだけを変えても、残り2つが引き戻す。

技術

アーキテクチャ、コード
インフラ、ツール

組織

チーム構造、文化
スキル、コミュニケーション

戦略

ビジネス目標、優先度
投資判断、ロードマップ

3つの変数に解析解はない。正解を計算するな。小さく動いて観測せよ。



なぜモダナイゼーションが必要なのか

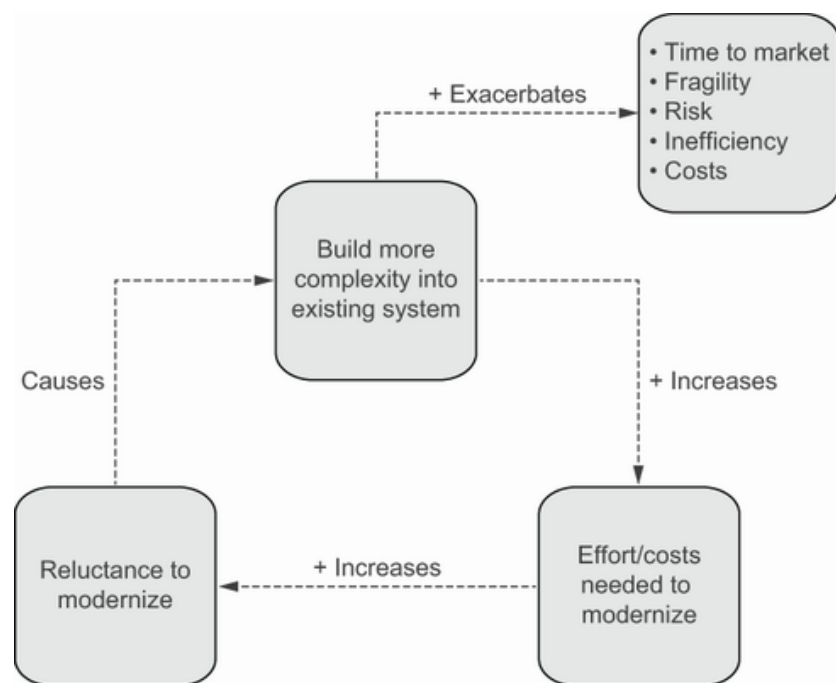


Figure 1.1 Why modernize? より引用

問い：今のシステムを維持し続けて、5年後も競争できるか？

私がこの問いを突きつけられたのは、翻訳作業の途中だった。

「改善したい」という意志はある。だが改善の速度が競合に追い越される速度に負けたら、意志は何の意味も持たない。

優秀なエンジニアは時間の42%を技術的負債に奪われている

(Stripe調査, 2018)

モダナイゼーションの4つの動機

競争力の維持

市場の変化に追従できない = 淘汰される。競合が3日で出す機能を、3ヶ月かけていては勝てない。

技術的負債の返済

負債は複利で増える。今日の「後で直す」は、来年の「直せない」になる。

開発者体験の向上

優秀なエンジニアは「成長できない環境」から離れる。レガシーと戦うだけの日々は、誰も望まない。

ビジネスアジリティ

新機能のリードタイムが長すぎると、市場機会を逃す。「技術的に難しい」が口癖になっていないか。

あなたの組織の痛みはどれか。まず自社の入り口に名前をつけることから始めよ



ビジネス目標がすべての起点

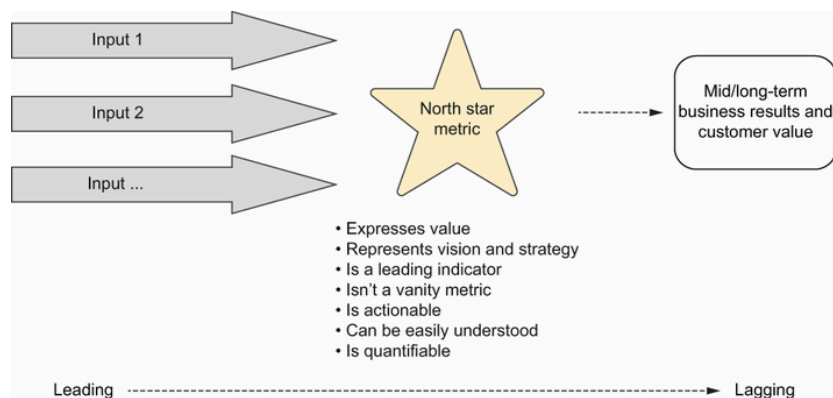


Figure 3.2 Aligning architecture modernization with business goals より引用

「技術を新しくしたい」はビジネス目標ではない

モダナイゼーションの正当化は、ビジネス目標との整合から始まる。経営層が理解できる言葉で語れなければ、予算も人も集まらない。

3つの問い: なぜ今やるのか？やらなかったら何が起きるか？成功したら何が変わるか？

技術的負債とは何か

Ward Cunninghamが1992年に提唱した比喻

将来の変更を困難にする、過去の技術的決定の蓄積。金融の負債と同様に「利子」がつく。ただし金融の負債と違い、技術的負債は意図せず発生し、残高が不明確なことが多い。

技術的負債の種類

種類	説明	例	対処法
意図的	短期目標のため意識的に受け入れる	MVP優先、後でリファクタ	返済計画を立てて追跡
無意識	知識不足から生じる	設計ミス、ベストプラクティス無視	コードレビュー、学習投資
環境的	外部要因から生じる	フレームワークの陳腐化	定期的な依存関係更新



技術的負債の複利効果

負債が増える理由

「後で直す」が永遠に来ず、負債の上に負債を重ねる。ドキュメントは陳腐化し、知識を持つ人は離職する。**負債は複利で増える**。しかも組織・法制度・業務フローは常に変化するため、何もしなくても負債は増え続ける。

負債の影響

新機能開発が遅延し、バグ修正が困難になり、オンボーディングコストが増加し、モチベーションが低下する。**全てが悪循環に入る**。AIで新規コードを高速に書いても、既存の負債が消えるわけではない——むしろ負債の上に新しいコードが積み上がり、問題を深くする。

システムは作られた瞬間から老化が始まる。投資し続けなければ、新規開発すらレガシーになる。



モダナイゼーションの銀の弾丸はない

「でも、マイクロサービスにすれば解決するのでは？」――この問いに正面から答えたい。

モダナイゼーションは成果が見えにくい。「マイクロサービス」と言えば経営層にも伝わるし、予算が通る。だがそれは「ダイエットしたい→サプリを買う」と同じ構造。本当に必要なのは生活習慣の変革であり、サプリは補助にすぎない。

失敗パターン

「マイクロサービスにすれば解決」「クラウドに移行すれば解決」「AIで全部自動化」――**手段が目的化**している。そもそも「レガシー」「モダン」という主語が大きすぎる。決済システムのレガシーと社内ツールのレガシーでは、対処法がまったく違う。

成功のアプローチ

ビジネス目標との整合、段階的な改善、組織と技術の同時変革。**目的から逆算する**姿勢が必要。バズワード的な二項対立（「〇〇 is Dead」「内製 vs 外注」）は、多様な領域を一括りにすることで**議論の粒度を下げ、本質を見失わせる**。

説明しやすい嘘より、説明しにくい真実を選ぶか。それがリーダーの仕事。



「説明しにくい真実」とは何か

経営層に通りにくいが、正しい判断

説明しやすい嘘	説明しにくい真実
「マイクロサービス化に2年かかります」	「まず半年かけて現状を理解します」
「新しいフレームワークで生産性2倍」	「今のコードを理解できる人を増やします」
「クラウド移行でコスト30%削減」	「運用を見直さないと移行しても変わりません」

なぜ「説明しにくい真実」を通すのが難しいか

成果が出るまでに時間がかかり、その間「何をしているか」が外から見えにくい。さらに「やらなかった場合の損失」は証明できず、競合が「銀の弾丸」で成功したように見えてしまう。

「半年後に失敗するプロジェクト」より「半年かけて基盤を作る」を選ぶか



見えないコストを見る力

人は「何ができるか」に注目し、「何を防いでいるか」を軽視する

機能的価値（新機能、画面、API）は目に見える。非機能要件（セキュリティ、コンプライアンス、可用性、属人化リスク）は目に見えない。だが、システムの長期的な存続を決めるのは後者のほう。

外部ベンダーへの支払いの正体

「自分たちで作れるのに、なぜ外部に払うのか？」という問いが出たとき、見落とされているものがある。外部への支払いには機能だけでなく、**リスクと責任の移転**という対価が含まれている。セキュリティアップデート、法令対応、24/365の運用――これらを自前で持つコストは、ライセンス費の何倍にもなりうる。

属人化という静かな爆弾

内製システム最大のリスクは属人化。「あの人がいないと動かない」状態は、人の流動性がある組織では時限爆弾になる。ドキュメントを書いても読まれない。知識が1人の頭に閉じている限り、その人の退職はシステム障害と同義。**持続可能性の観点を欠いた内製は、最も高コストな選択**になりうる。

「いくらかかるか」ではなく「何のリスクを誰が引き受けるか」で判断する



"Success breeds inertia, and the greater the past success, the greater the inertia"

— Simon Wardley

成功は惰性を生む。過去の成功が大きいほど、惰性も大きくなる

では、判断はどんな情報に基づくべきか？

ここまでの話

AI時代には「何を作るか」「なぜ作るか」を決める力が重要になる。技術が楽になると人間の問題がボトルネックとして露わになり、技術的負債は複利で増えるが銀の弾丸はない。

だが、問いが残る

「判断する」と言っても、判断には情報が要る。技術だけ見ても全体像は見えない。組織を無視して技術を変えても、組織が引き戻す。

答えは「技術と組織を同時に見ること」。次のセクションでは、その視点——**ソシオテクニカル**——を紹介する。



2. ソフトウェアと組織を 同時に設計する

ソシオテクニカルとは何か

Socio（社会的・組織的） + Technical（技術的） = ソシオテクニカル

1960年代にタビストック研究所で生まれた概念。ソフトウェアアーキテクチャと組織構造は切り離せない。どちらか片方だけを変えても、真のモダナイゼーションは実現できない。

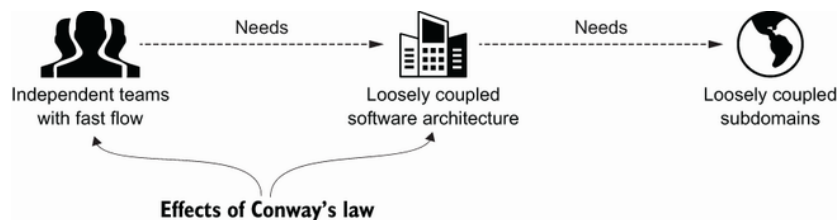


Figure 2.1 Sociotechnical architecture より引用

技術だけ変えても

組織の壁でデプロイが止まる

組織だけ変えても

システムの依存関係でチームが自律できない

なぜ同時設計ができないのか

「技術と組織を同時に設計すべき」は正しい。だが現実にはできていない。

縦割りの壁

技術はエンジニア、組織は人事・経営の領域。「アーキテクチャの議論に人事を呼ぶ」という**発想自体がない**。

時間軸の違い

技術変更は数週間～数ヶ月、組織変更は数ヶ月～数年。McKinsey（2023）の調査では、デジタル変革の70%が目標未達であり、最大の原因は「技術と組織の変革速度の不一致」と報告されている。**同期が取れない**ため、どちらかが置いていかれる。

政治的コスト

組織変更は人の感情が絡み、「あなたのチームを解体します」は言いにくい。**技術だけ変える方が楽に見える**。

この壁を越える権限がないなら、その権限を獲りに行け



なぜソシオテクニカルが重要なのか

レガシーの問題は、技術だけでなく組織・文化・プロセスに根ざしている。

関連する概念

概念	説明
コンウェイの法則	組織構造がシステム設計に反映される
逆コンウェイ戦略	望ましいアーキテクチャに合わせて組織を設計する

技術だけ変えても組織が追いつかない。組織だけ変えても技術が足を引っ張る



翻訳で体験したソシオテクニカル

この本を5人で翻訳した経験は、まさにソシオテクニカルの実験場だった。

技術的な課題

翻訳ツール、用語集の管理、Gitでの原稿管理。200以上のコミットを重ねて品質を磨いた。

組織的な課題

5人の「解釈」の違い。同じ英語を違う日本語に訳す。章をまたぐと整合性が崩れる。**暗黙の前提が異なっていた。**

結果として起きたこと

ツールだけ揃えても訳文は揃わない。
「同じ言葉を同じ意味で使う」という組織的合意――まさにユビキタス言語――が必要だった。

5人のチームでも起きる。あなたのチームでも必ず起きている。見えていないだけだ



コンウェイの法則

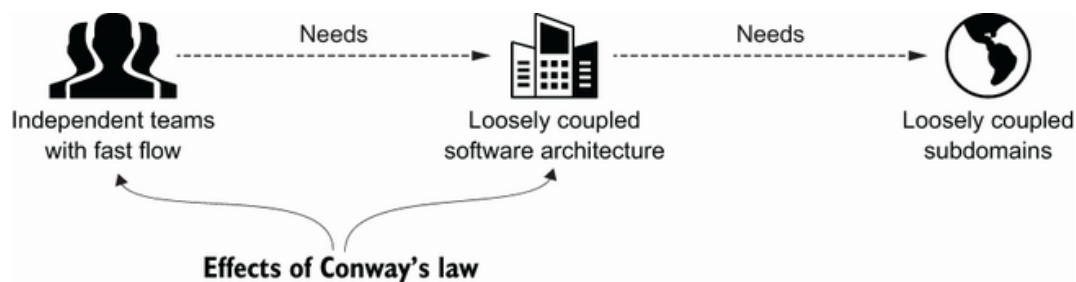


Figure 2.1 Conway's Law より引用

"Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations" — Melvin Conway (1968)

システムを設計する組織は、その組織のコミュニケーション構造をコピーした設計を生み出すことを余儀なくされる。

疎結合なドメイン境界が、疎結合なアーキテクチャを可能にする

- チーム構造を変えずにアーキテクチャだけ変えても失敗する
- 「逆コンウェイ戦略」：望むアーキテクチャに合わせてチームを編成する

組織の形がシステムの形になる。これは好むと好まざるとに関わらず起きる。ならば利用せよ。

コンウェイの法則が働く理由

「組織を変えずにアーキテクチャだけ変えればいい」――これが最もよく聞く反論。だが、なぜそれが通用しないのか。

コミュニケーションコスト

チーム内の会話は低コストだが、チーム間は調整・承認・ドキュメントが必要で高コスト。結果として、**高コストな境界でモジュールが切れる**。

責任の分界点

「ここから先は別チームの責任」という境界が**インターフェース境界**になる。責任が曖昧だと**密結合の温床**になる。

暗黙知の共有範囲

チーム内では文脈が共有されるが、チーム外には明示的なAPIが必要。**共有文脈の境界がモジュール境界**になる。

コンウェイの法則は「選択肢」ではなく「物理法則」。抗うより利用せよ

コンウェイの法則を味方にした先に何がある？

組織とアーキテクチャを揃えたとして、その成功をどう測るか？

「デプロイ頻度が上がった」だけで満足していいのか。速度だけを追うと、品質・安全性・チームの持続可能性が犠牲になりかねない。モダナイゼーションの成果には、**多面的な物差し**が必要。

→ Jon Smartが提唱した **BVSSH (Better Value Sooner Safer Happier)** フレームワークで、モダナイゼーションの「本当の成功」を測る

BVSSH: モダナイゼーションの成果指標

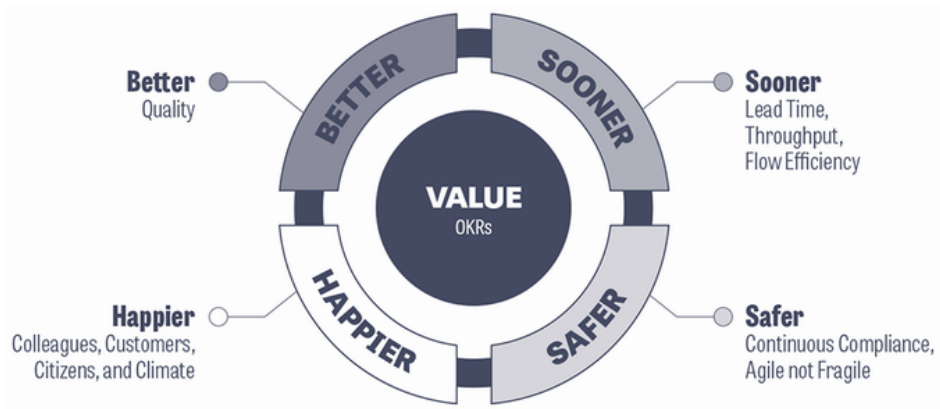


Figure 1.3 Better Value Sooner Safer Happier (Source: Smart et al., Sooner Safer Happier [IT Revolution, 2020]) より引用

Jon Smartが提唱したBVSSHフレームワーク

Better — 品質向上、手戻り削減

Value — ビジネス成果

Sooner — 時間短縮

Safer — セキュリティ

Happier — 従業員とステークホルダーの満足度

BVSSHの5つの指標

「速くなった」だけでは不十分。 Soonerだけ伸ばした結果、品質低下・インシデント3倍・離職率上昇という事例がある。

指標	危険な兆候
Better	速くなったが品質が下がった
Value	機能は増えたが使われない
Sooner	頻度は上がったがバグも増えた
Safer	速度優先でセキュリティ軽視
Happier	速くなったが燃え尽きた

5つの指標は互いにトレードオフの関係になり得る。1つだけ伸ばしても、他が崩れれば全体としては後退する。



独立した価値ストリーム (IVS)

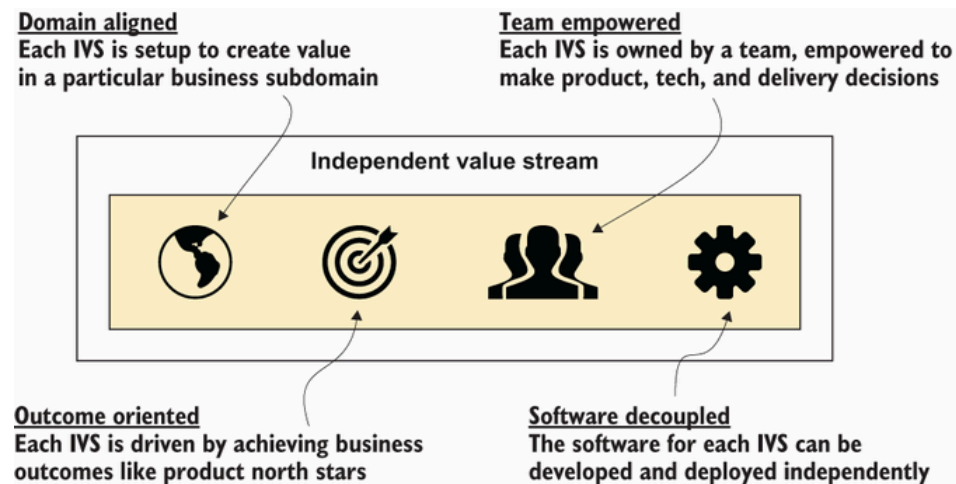


Figure 1.5 The four key characteristics of an independent value stream より引用

モダンアーキテクチャの基本単位「Independent Value Stream」

4つの特性

1. **ドメイン整合**: 特定のビジネス領域に焦点
2. **チーム自律**: 製品・技術・デリバリーの決定権
3. **成果志向**: 機能数ではなくビジネス成果で測定
4. **疎結合**: 独立してデプロイ可能

"Teams aligned to domains become owners of their product's destiny"
ドメインに整合したチームは、プロダクトの運命の所有者になる

IVSの4つの特性を深掘り

ドメイン整合

チームが担当する範囲がビジネスドメインと一致している。「決済チーム」は決済のすべてを知っている。

→ 認知負荷の軽減、専門性の深化

成果志向

「機能を何個作ったか」ではなく「顧客にどんな価値を届けたか」で測る。

→ 正しいものを作る動機づけ

チーム自律

技術選定、リリース判断、優先順位付けをチームが決められる。上位承認を待たない。

→ "You build it, you run it" — 責任が信頼性を生む

疎結合

他チームに依頼しなくてもデプロイできる。共有データベースや共有ライブラリへの依存が少ない。

→ デリバリー速度の向上



IVSを実現するための条件

IVSは「宣言するだけ」では実現しない

4つの特性を満たすには、技術的・組織的な前提条件がある。

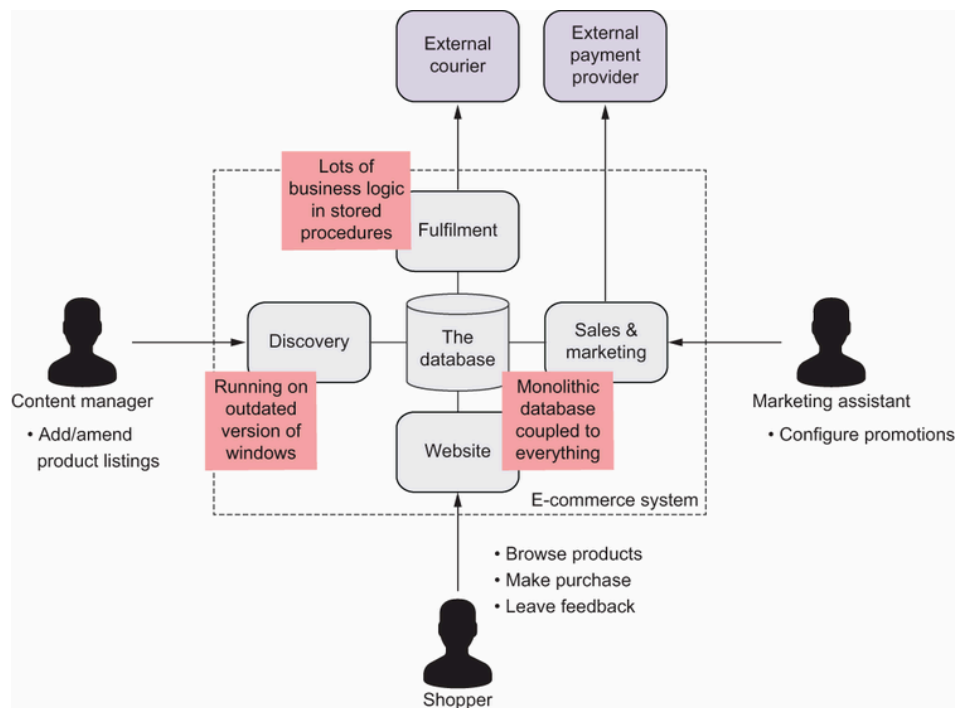
前提条件のチェックリスト

特性	前提条件	よくある障害	対処法
ドメイン整合	ドメイン境界が明確	曖昧な責任分担	イベントストーミングで可視化
チーム自律	権限委譲の文化	マイクロマネジメント	段階的に権限を委譲、成功体験を積む
成果志向	ビジネス指標の可視化	技術指標だけで評価	OKRでビジネス成果に紐付け
疎結合	APIファースト、CI/CD	共有DB、モノリス	Strangler Figで段階的に分離

「IVSにしたい」の前に「IVSにできる状態か」を確認する



リスニングツアー



組織全体からインサイトを収集する構造化されたアプローチ

モダナイゼーションを始める前に、現状を深く理解することが重要。

実施内容

- **対象者:** 開発者、PM、ビジネスステークホルダー、運用チーム
- **質問例:** 最も痛みを感じている部分は？なぜ変更が難しいのか？

効果

- 問題の全体像を把握
- 組織全体のバイイン（賛同）を獲得
- 優先順位付けの材料を収集

Figure 4.4 A visual summary from a listening tour より引用

リスニングツアーで役割別に質問する

開発者への質問

- デプロイ頻度は？ブロッカーは何？
- 最も触りたくないコードは？
- チーム間の依存で困ることは？

PMへの質問

- 機能リリースのリードタイムは？
- 優先順位の変更にどう対応している？
- ステークホルダーからの不満は？

ビジネス側への質問

- 競合に勝てないと感じる領域は？
- 「もっと早くできないの？」と思うのは？
- 技術チームへの期待と現実のギャップは？

運用チームへの質問

- 最も障害が多いシステムは？
- 夜間対応が必要になる原因は？
- 「直したい」と思っても直せないものは？



リスニングツアーで収集した情報を分析する

「聞くこと」自体が変化の始まり

リスニングツアーは情報収集だけでなく、「経営層が現場の声を聞いている」というメッセージになる。これがモダナイゼーションへのバイイン（賛同）を生む。

収集した情報の整理

分類	内容	次のアクション
痛み	現場が困っていること	優先度の判断材料
願望	「こうなったらいい」	ビジョンの素材
政治	誰が何を守りたいか	障害の予測
知恵	現場ならではの知識	設計への反映

聞いた内容は必ず**匿名化して共有**する。「誰が言ったか」ではなく「何が問題か」に焦点を当てる。



マッピングツアー

システムとビジネスの関係を可視化

リスニングツアーで得た情報を整理し、マップとして表現する。

マッピングの対象

技術的マッピング

- システム間の依存関係
- データフロー
- 技術スタック

組織的マッピング

- チーム構造
- 責任範囲
- コミュニケーションパス

これらのマップが、コンウェイの法則を可視化し、逆コンウェイ戦略の立案に使える。



マッピングツアーで依存関係を可視化する

「誰が誰に依存しているか」を明らかにする

依存関係を可視化すると、ボトルネックと変更の影響範囲が見えてくる。

描くべきマップ

マップ	内容	発見できること
システム依存図	どのシステムがどのAPIを呼んでいるか	変更の影響範囲
データフロー図	データがどこからどこへ流れるか	ボトルネック、単一障害点
チーム責任マップ	どのチームがどのシステムを担当するか	オーナーシップの曖昧さ

ポイント

- 完璧を目指さない。80%正しければ十分
- 「これは誰に聞けばいいの？」という質問自体が価値ある発見



マッピングツアーでギャップを発見する

技術マップと組織マップを重ねると、ギャップが見える

よくあるギャップ

- 1つのシステムを複数チームが触る
- 1つのチームが関連のない複数システムを担当
- 依存先チームとのコミュニケーションパスがない
- 「誰がオーナーかわからない」システム

ギャップが示すこと

- コンウェイの法則が働いていない箇所
- 逆コンウェイ戦略で改善できる箇所
- モダナイゼーションの優先度が高い領域

完璧なマップは要らない。来週、ホワイトボードに依存関係を1つ描け。会話が始まる



プロダクトタクソミー

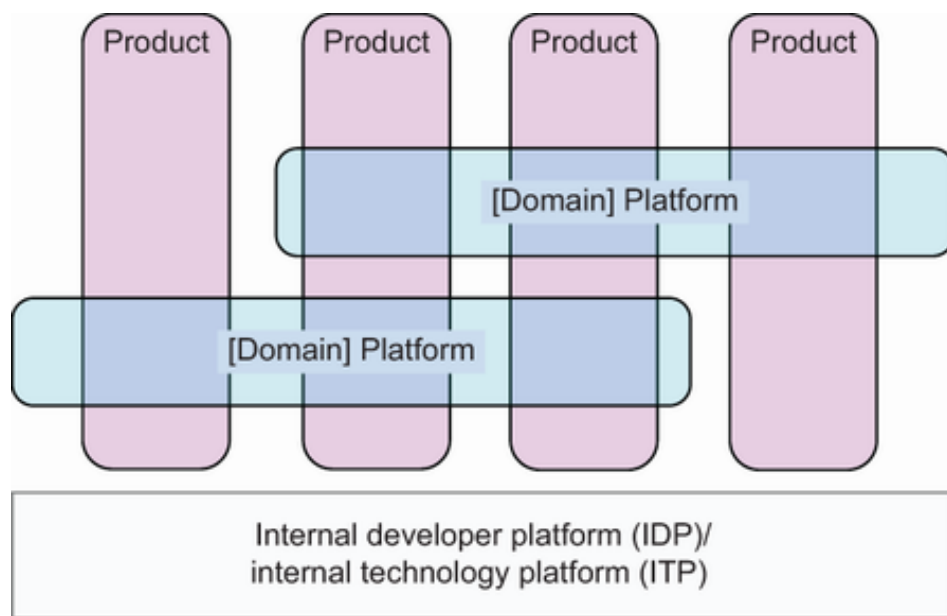


Figure 6.6 A product taxonomy より引用

製品とサービスを体系的に分類・整理する手法

モダナイゼーションの対象を明確にするために、組織が持つ製品・サービスを整理。

分類の軸

ビジネス軸

顧客セグメント、収益モデル、市場での位置づけ

技術軸

技術スタック、アーキテクチャパターン、依存関係

地味な仕事こそモダナイゼーションの土台

リスニングツアー、マッピングツアー、プロダクトタクソノミー

これらはSNSでバズらない。カンファレンスのトレンドにもならない。「うちもやってます」とは言いにくい地味な作業。そして四半期の評価に書きにくい。

派手な施策

「マイクロサービス導入」「Kubernetes移行」「新フレームワーク採用」——発表しやすい、評価されやすい。

地味だけど重要な作業

現場の声を丁寧に聞く、依存関係を可視化する、製品を整理・分類する——発表しにくい、土台になる。

派手な技術選定より、地味な現状把握。それがモダナイゼーションの第一歩。



逆コンウェイ戦略の実践

望ましいアーキテクチャに合わせて組織を設計する

コンウェイの法則を逆手に取る戦略。

ステップ

1. 目標アーキテクチャの定義: 疎結合で独立デプロイ可能な構造
2. 必要なチーム構造の設計: 目標に合わせたチーム境界
3. 段階的な移行: 一度にすべてを変えない
4. 継続的な調整: フィードバックに基づく改善

権限なき責任は拷問。チームに責任を求めるなら、権限もセットで渡せ。

責任だけ押し付けて権限を握り続けるリーダーは、無意識にモダナイゼーションを妨害している



構造とプロセスの誤謬



逆コンウェイ戦略は強力だが、落とし穴がある。組織図を書き換えるだけでは何も変わらない。

よくある失敗

組織図を変えただけ、新しいプロセスを導入しただけ、ツールを変えただけ——**形だけ変えて満足する。**

本当に必要なこと

文化の変革、スキルの向上、技術的な改善、継続的な実践。**構造と中身の両方**を変える必要がある。

箱を入れ替えても、中身が同じなら結果は同じ。形だけの変革は変革ではない

出典: 日経BP 日本経済新聞出版

なぜ「構造を変えるだけ」では失敗するのか



宇田川元一著（2024）

「構造的無能化」（宇田川元一） — 組織の断片化が進み、思考の幅と質が制約され、目先の問題解決を繰り返して疲弊していく現象。成功した大企業ほど陥りやすい。

3つの症状とモダナイゼーションへの影響

症状	説明	モダナイゼーションへの影響
断片化	分業化しすぎて縦割りに	チーム間の壁、サイロ化
不全化	変化を察知して自ら動けない	「誰かが決めてくれる」待ち
表層化	場当たりの対応しか取れない	銀の弾丸への期待

成功した組織ほどこの罠にはまる。だからこそ、構造を意識的に壊し続ける必要がある

"Well-designed domain boundaries reduce dependencies and maximize team autonomy"

適切に設計されたドメイン境界は
依存関係を減らし、チームの自律性を最大化する

同時設計をどう実現するか？

ここまでの話でわかったこと

組織とアーキテクチャは同時に設計しなければ、片方が足を引っ張る。コンウェイの法則は物理法則のように避けられず、独立した価値ストリーム（IVS）が目指すべき姿である。

だが、暗黙の前提が邪魔をする

組織の中には「言語化されていない前提」が無数にある。アーキテクトが一人で設計書を書いても、現場の暗黙知は反映されない。ビジネス側に聞いても技術的制約が見えない。

この暗黙知を可視化する手法がある。それが**イベントストーミング**。

3. ビジネスを可視化する 協働的手法

イベントストーリーミングとは



Figure 7.1 EventStorming timeline より引用

Alberto Brandoliniが考案した、ビジネスドメインを探索するためのワークショップ手法。「正確さより発見を優先する」という哲学が特徴。

基本的な要素

付箋の色	意味
オレンジ	ドメインイベント（過去形）
青	コマンド（アクション）
黄色	アクター/ユーザー
ピンク	外部システム
濃いピンク	ホットスポット（問題）

イベントストーリーミングの記法

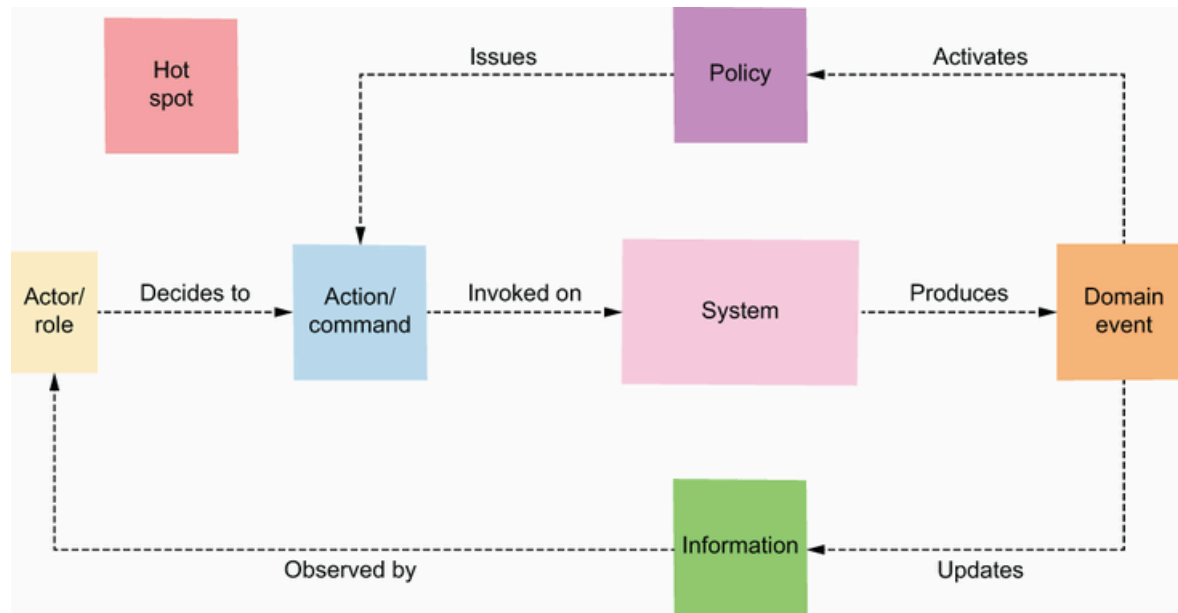


Figure 7.3 People, systems and hotspots より引用

記法の意味

- オレンジの付箋: 過去形で書く「～された」
- 青の付箋: 命令形で書く「～する」
- 矢印: イベントの流れ、因果関係
- ピンクのホットスポット: 曖昧さや問題点をマーク

ポイント

最初は「正しさ」を気にせず、とにかく貼り出す。発見は混乱の中から生まれる。

イベントストーリーミングの進め方



Figure 7.1 Big Picture EventStorming より引用

Phase 1: カオス

参加者全員がドメインイベントを時系列で貼り出す。この段階では議論せず、とにかく可視化。

"Chaos is a feature, not a bug"

Phase 2: タイムライン

イベントを時間軸で整理。矛盾や抜けを発見していく。

Phase 3: 問題の発見

ホットスポット（問題点、曖昧な箇所）をマーク。ここが改善のチャンス。

Phase 4: 境界の特定

自然なドメイン境界を見つけ、サブドメインを定義する。



イベントストーミングの価値

なぜトップダウンの設計より効果的なのか

- **多様な視点の統合:** 開発者、ドメインエキスパート、UXデザイナーが同じ場に
- **暗黙知の可視化:** ドキュメントには書かれていない知識が浮かび上がる
- **共通理解の形成:** 認識のズレがその場で解消される
- **設計の民主化:** 「アーキテクトだけが知っている」状態を脱却

従来の「アーキテクトが設計して渡す」モデルでは、現場の知識が設計に反映されにくい。イベントストーミングは、**設計プロセス自体を協働的にする**という発想の転換。



翻訳チームで起きたイベントストーリーミング

翻訳チームで起きた問題は、イベントストーリーミングで解決される典型的パターンだった。

暗黙の前提がズれていた

5人が同じ英語を違う日本語に訳す。
「これはこういう意味だと思っていた」
——その前提が**暗黙のまま**放置されていた。

可視化で解決した

用語集を作り、各章担当者が集まって認識合わせ。「この文脈でのこの用語の意味」を**全員で可視化**した。イベントストーリーミングと同じ原理。

これはBounded Contextの実践

「全体で統一」ではなく「境界内で一貫」。章ごとに文脈が違うことを認め、境界を明示した。次のDDDセクションで詳しく説明する。

翻訳で最も難しかったのは英語ではない。著者の思考を追体験することだった。

「行間」を翻訳する——暗黙知の発見

ドキュメントに書いてあることと、著者が言いたいことは違う。原著の「行間」を読む必要があった。文脈を知らないと誤訳する。「なぜこの章がこの順番なのか」を理解して初めて訳せた。

翻訳の現場

著者Nick Tuneが「なぜBounded Contextの章をイベントストーリーミングの後に置いたのか」——その意図を理解しないと、章の繋がりが訳せない。

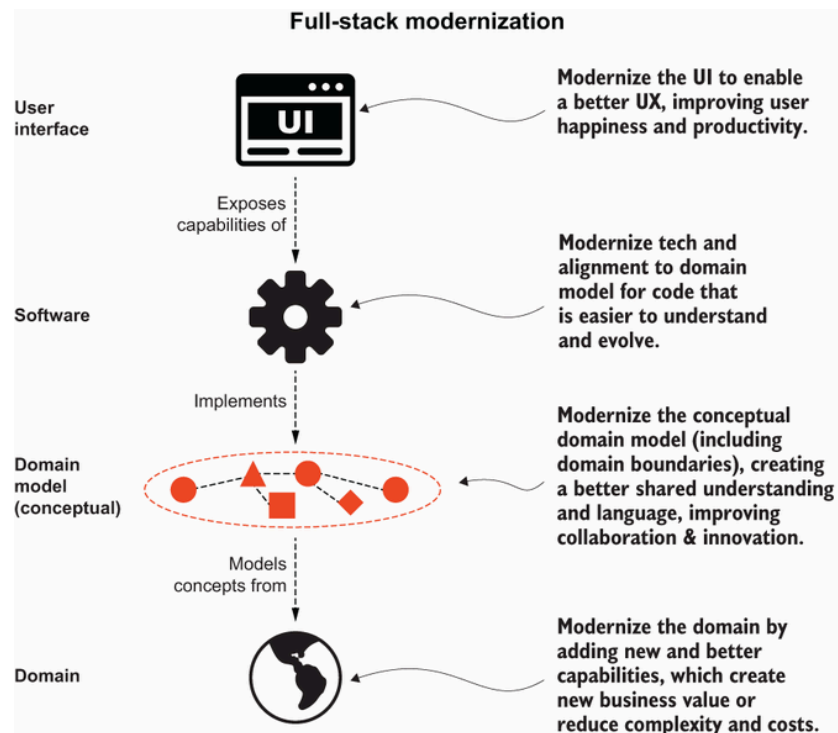
ソフトウェア開発の現場

仕様書に書いてあることと、顧客が欲しいものは違う。既存コードを読むだけでは設計意図がわからない。「なぜ」が共有できているかが品質を決める。

暗黙知を可視化する——翻訳でもソフトウェアでも、イベントストーリーミングの原理は同じ



イベントストーリーミングの成果を設計に活かす



イベントストーリーミングで得られたもの

- ビジネスフローが可視化された
- ホットスポット（問題箇所）が特定された
- チーム間の暗黙知が共有された

次の問い: 「この可視化されたビジネスフローを、どうシステム設計に落とし込むか？」

→ **DDD**の概念（Bounded Context、コンテキストマッピング）で整理する

Figure 8.1 Domain discovery through EventStorming より引用

ドメイン境界を発見する

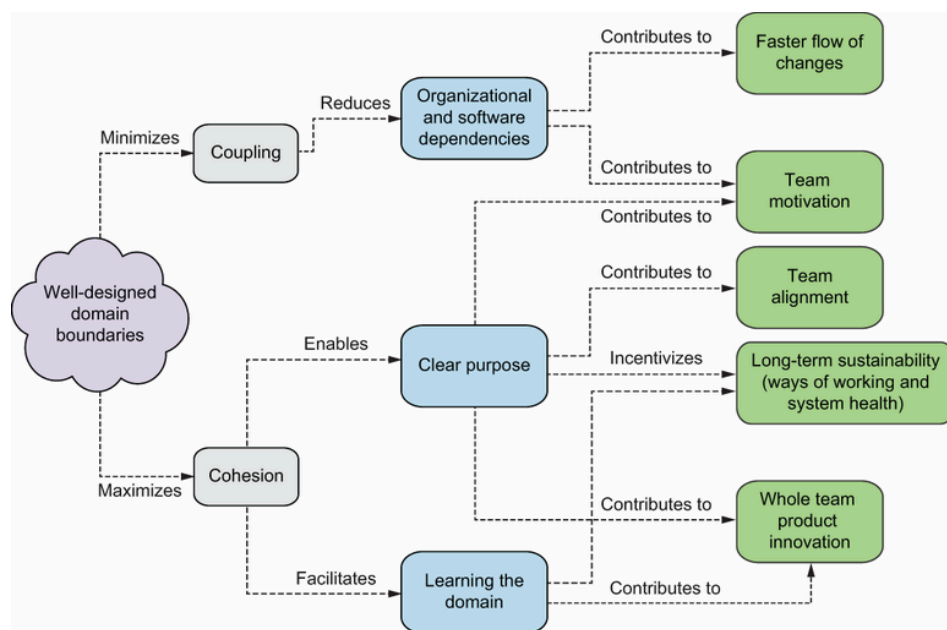


Figure 9.1 Well-designed domain boundaries reduce dependencies より引用

境界を定義する原則

「正しい」唯一の境界は存在しない。境界は組織の目標に奉仕するために存在する。

一緒に変わるものは、一緒に置く。変更の波及範囲が境界の外に漏れるなら、境界の引き方が間違っている。

依存関係のコスト評価

$$\text{Pain} = \text{Strength} \times \text{Volatility} \times \text{Distance}$$

- **Strength:** 依存の強さ
- **Volatility:** 変更頻度
- **Distance:** チーム間の距離

強く依存し、よく変わり、遠いチームとの関係が最もコストが高い。

Bounded Contextとは何か

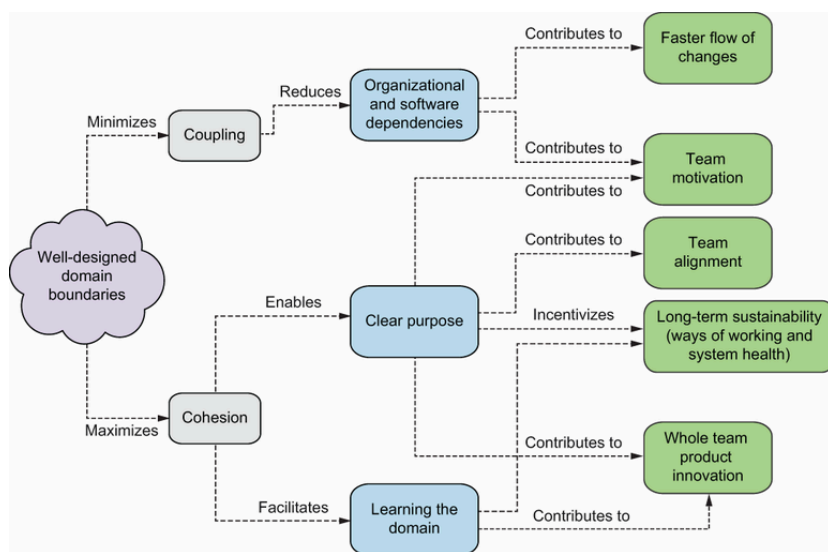


Figure 9.2 Bounded Context and its relationships より引用

問い：システム全体で「顧客」の定義を統一すべきか？

Bounded Context = 同じ言葉が同じ意味を持つ範囲

Eric Evansが2003年のDDDで提唱した中核概念。この範囲内では、この言葉はこの意味という明確な境界線。モデルの適用範囲を限定することで、矛盾なくドメインを表現できる。

なぜ重要か？ → 同じ言葉でも、文脈によって意味が異なるから。

なぜ「統一」ではなく「境界」なのか

例：「顧客」という言葉

コンテキスト	「顧客」の意味
営業	商談中の見込み客（連絡先、商談ステージ）
サポート	問い合わせをしてきた人（チケット履歴）
請求	支払い義務のある法人/個人（請求先住所）
マーケティング	セグメント、ペルソナ（行動履歴）

これを無視して「統一顧客マスタ」を作ると → **あらゆる属性を持つ巨大テーブル**になり、変更のたびに全部門に影響。

Bounded Contextが解決する問題

統一モデルの問題

全員の要求を満たそうとして肥大化し、1箇所の変更が全体に波及。「誰のためのモデルか」が曖昧になり、**チーム間の調整コストが爆発**する。

Bounded Contextのアプローチ

各コンテキストは必要なモデルだけ持ち、変更が局所化される。「このコンテキストの責任者」が明確で、**チームが自律的に動ける**。

Bounded Contextは「分断」ではなく「明確化」。境界を認めることで、かえって連携しやすくなる。



翻訳で証明されたBounded Context

翻訳チームで起きた用語論争は、Bounded Contextの必要性を身をもって証明した。

例：「Bounded Context」の訳語

「境界づけられたコンテキスト」で統一するか、初出のみ日本語を添えてあとは英語表記にするか。5人の翻訳者で議論が割れた。

解決策：用語集で「境界」を引いた

19の中核用語について「この表記、この意味」を明文化した。まさに**翻訳チーム自体がBounded Contextを実践**していた。

Bounded Contextは理論ではない。翻訳者5人の現場で必然的に生まれた解決策だった。

コンテキストマッピング

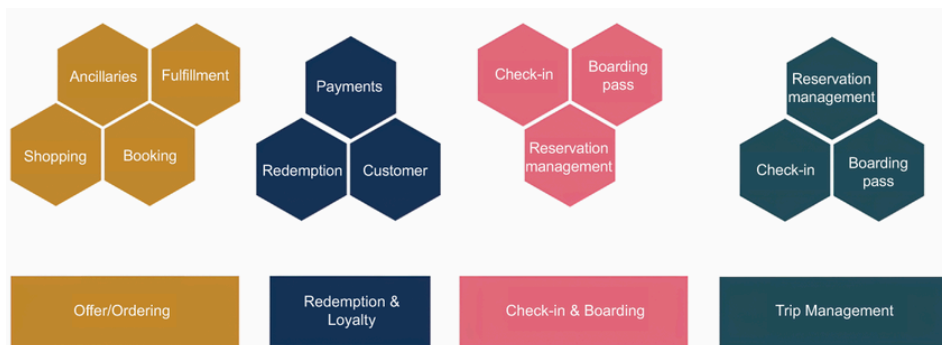


Figure 9.12 A context map より引用

Bounded Context同士の連携パターンを可視化する手法（Eric Evans, DDD）

Bounded Contextは独立して存在するわけではない。他のコンテキストとどう連携するかを明示的に設計する必要がある。

Team Topologiesのインタラクションモードと対応する概念であり、チーム間の関係性を技術的に表現したもの。

コンテキストマッピングの代表的なパターン

パートナーシップ (Partnership)

両チームが対等な立場で協力し、互いの成功が相手の成功に依存する関係。共同で開発計画を立て、API変更は両者で合意する。

特徴: 調整コストは高いが柔軟性も高い。片方だけが恩恵を受ける関係では機能しない。

例: 決済チームと注文チームが共同で新機能を開発

カスタマー・サプライヤー (Customer-Supplier)

上流チーム（サプライヤー）が下流チーム（カスタマー）のニーズに対応する。下流の要望が上流の優先度に影響を与える。

特徴: 上下関係が明確だが、下流の声が届く仕組みがある。上流が下流を無視し始めると崩壊する。

例: 基盤チーム（上流）と機能チーム（下流）

順応者 (Conformist)

下流が上流のモデルをそのまま受け入れ、上流に影響を与える力がない場合に採用。翻訳コストをかける余裕がない、または上流のモデルが十分に良い場合。

特徴: 実装は簡単だが、上流の変更に振り回される。戦略的に選択すべき。

例: 外部決済API（Stripe等）をそのまま使う

腐敗防止層 (Anti-Corruption Layer)

外部システムの影響から自チームを守る翻訳層。自分たちのドメインモデルを汚染させないための防御壁。

特徴: 実装コストはかかるが、自律性を確保できる。レガシーシステムとの統合に有効。

例: 古い基幹システムのデータを変換するアダプター

コンテキストマッピングで関係性を設計する

「繋がり方」を設計しないと、最悪の形で繋がる

Bounded Contextを発見しただけでは不十分。コンテキスト間の**関係性**を意図的に設計しなければ、暗黙の依存関係が蔓延する。

パターンの選択基準

パターン	採用する状況	避けるべき状況
パートナーシップ	両チームの成功が相互依存	片方だけが恩恵を受ける
カスタマー・サプライヤー	明確な上流/下流がある	双方向の影響がある
腐敗防止層 (ACL)	外部システムから保護したい	自チームの負担が大きすぎる

境界は自然発生しない。アーキテクトが意志を持って「引く」ものだ

境界を守る方法の一つ——型システム

ここまでの話

イベントストーリーミングでドメインを発見し、Bounded Contextで境界を定義し、コンテキストマッピングで関係性を設計した。

余談：境界をコードでも守る方法がある

ドキュメントに書いても破られる。コードレビューで指摘しても漏れる。そこで**コンパイラに守らせる**という選択肢がある。関数型ドメインモデリング（Scott Wlaschin）が提唱する **Make Illegal States Unrepresentable**——不正な状態を型レベルで表現不可能にするアプローチ。

境界は「約束」で守るな。「型」で守れ。



"Loosely coupled domain boundaries are not optional—they are essential"

疎結合なドメイン境界はオプションではない。それは必須である

境界をシステムとチームに実装する

境界が見えた。だが見えただけでは意味がない。

イベントストーリーミングで得られたものービジネスの全体像、ドメイン境界の候補（Bounded Context）、チーム間の暗黙知、型で境界を守る選択肢。

問いが変わる

「境界に合わせてチームをどう設計するのか？」

Bounded ContextとTeam Topologiesを対応させ、コンテキストマッピングとインタラクショモードを揃える。DDDとTeam Topologiesを統合的に活用する。



4. ドメイン駆動設計と チーム設計の統合

DDDとTeam Topologiesの対応関係

なぜ2つを統合的に使うのか？

DDDはソフトウェアの境界を定義し、Team Topologiesはチームの境界を定義する。両者を一致させることで、コンウェイの法則が味方になる。

DDD概念	Team Topologies対応
Bounded Context	Stream-aligned Team の責任範囲
Core Domain	最も重要なStream-aligned Team
Supporting Subdomain	Enabling Team / Complicated Subsystem Team
Generic Subdomain	Platform Team / 外部調達
Context Mapping	Team Interaction Modes

「きれいに分離できない」という反論が出る。完全な分離は幻想。依存の方向と強さを制御することが本質



戦略的ITポートフォリオ

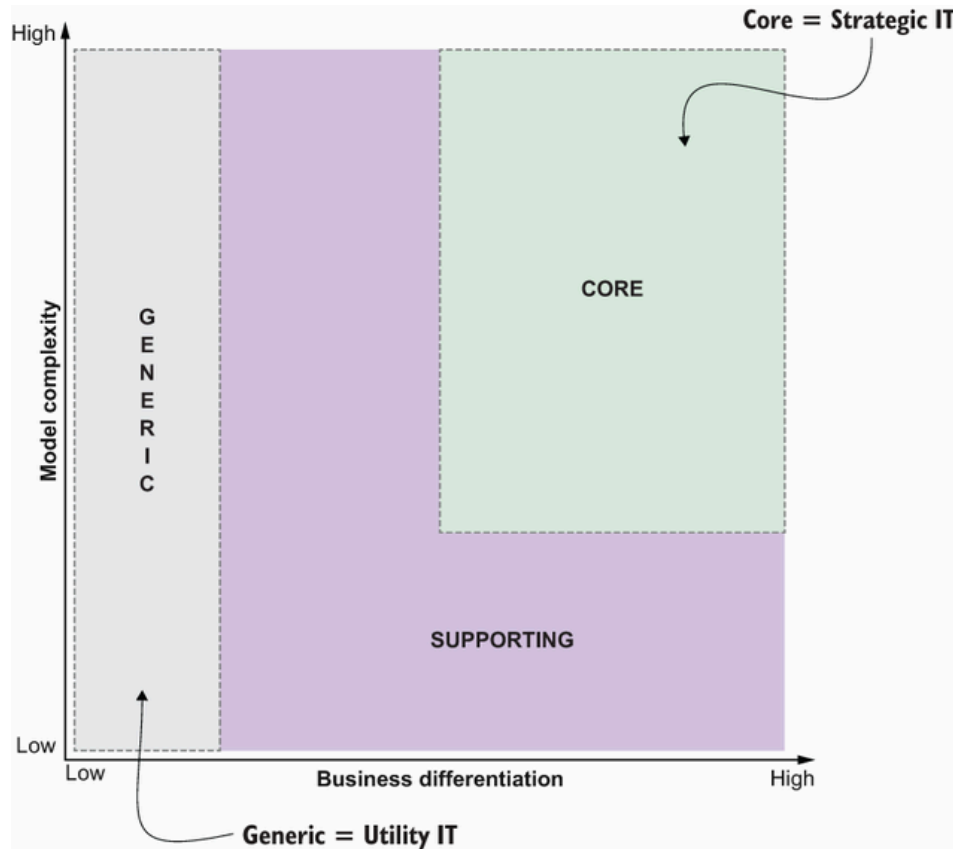
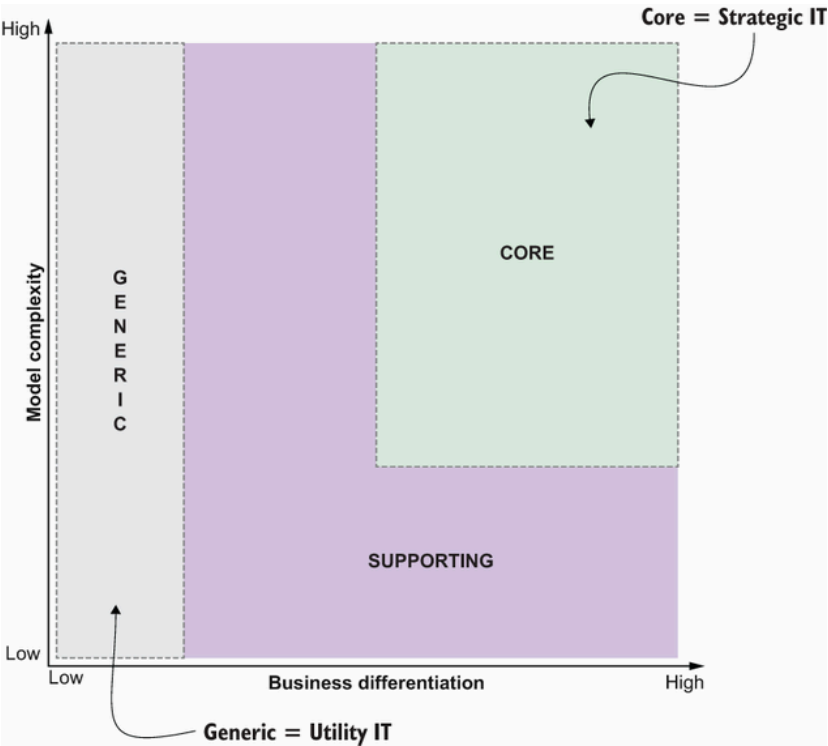


Figure 10.4 Strategic IT portfolio assessment より引用

すべてのシステムを一律に扱うのは戦略ではない

ポートフォリオ全体を俯瞰し、どのシステムに投資し、どのシステムを置き換え、どのシステムを退役させるかを判断する。Core Domain Chartと組み合わせることで、投資の優先順位が明確になる。

Core Domain Chart



問い：すべてのドメインに同じリソースを投入すべきか？

答え：No。戦略的に投資先を選ぶ必要がある。

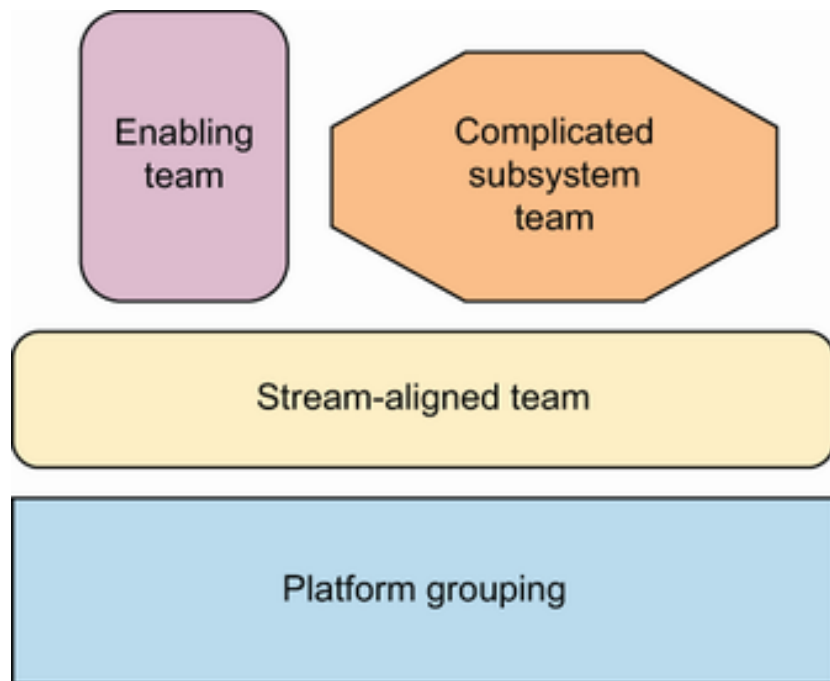
分類	差別化	複雑度	投資戦略
Core	高	高	重点投資、内製
Supporting	低	高	効率重視、外部委託も
Generic	低	低	購入/SaaS

X軸: ビジネス差別化、Y軸: モデル複雑度

Figure 10.5 The Core Domain Chart より引用



Team Topologies とは



Matthew SkeltonとManuel Paisが提唱。「高速なフローを持続可能に実現する」ためのチーム設計フレームワーク。

4つのチームタイプ

Stream-aligned — 価値ストリームをEnd-to-Endで担当

Platform — 共通基盤を提供

Complicated Subsystem
高度な専門知識が必要な領域を担当

Enabling — 他チームのスキル向上を支援

Figure 11.3 The four team types in Team Topologies より引用

Stream-alignedチームの設計原則

Stream-alignedチームが「主役」

Team Topologiesの核心は、Stream-alignedチームが価値を届けることに集中できるよう、他の3タイプがサポートする構造。

Stream-alignedチームに必要な条件

条件	説明	なぜ重要か
End-to-End責任	設計からデプロイまで	他チームへの依存を減らす
ビジネス成果へのオーナーシップ	機能数ではなく価値	正しいものを作る動機
認知負荷の限界内	担当範囲が大きすぎない	持続可能なペース

来週、1つの機能リリースに必要な調整先を数えてみる。その数があなたの組織の現在地だ



3つのインタラクションモード

Step 1: Teams work closely on a new domain/challenge with fuzzy boundaries

Step 2: Boundaries start to emerge so teams can work more independently

Step 3: Clear boundaries are established and teams can work largely independently

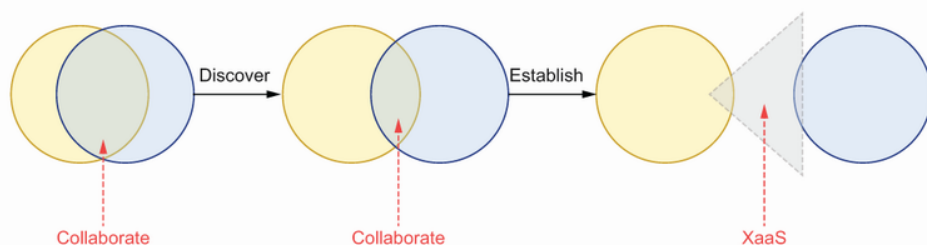


Figure 11.5 The three interaction modes in Team Topologies より引用

チーム間の関わり方を明確に定義する

Collaboration

高帯域幅での協働。コスト高だが発見多い。

X-as-a-Service

セルフサービスAPI経由での利用。

Facilitating — Enablingチームが他チームの能力向上を支援

インタラクションモードの進化

典型的な進化パターン

[発見期] Collaboration 密な協働 高コスト	→	[移行期] Facilitating スキル移転 中コスト	→	[成熟期] X-as-a-Service セルフサービス 低コスト
--	---	--	---	--

なぜ進化が重要か

Collaborationは持続不可能——2チームが常に密に連携すると両方の速度が落ちる。**X-as-a-Service**は最初から無理——境界が不明確な段階でAPIを固定すると後で苦しむ。**Facilitating**は橋渡し——知識を移転し、自律性を高める。

チーム間の関わり方に「卒業条件」を設定せよ。期限なきCollaborationは共依存になる



DDDとTeam Topologiesの統合

ドメイン境界 = チーム境界

DDDで発見したサブドメインの境界と、Team Topologiesで設計するチーム境界を一致させる。これが「逆コンウェイ戦略」の実践。

統合の効果

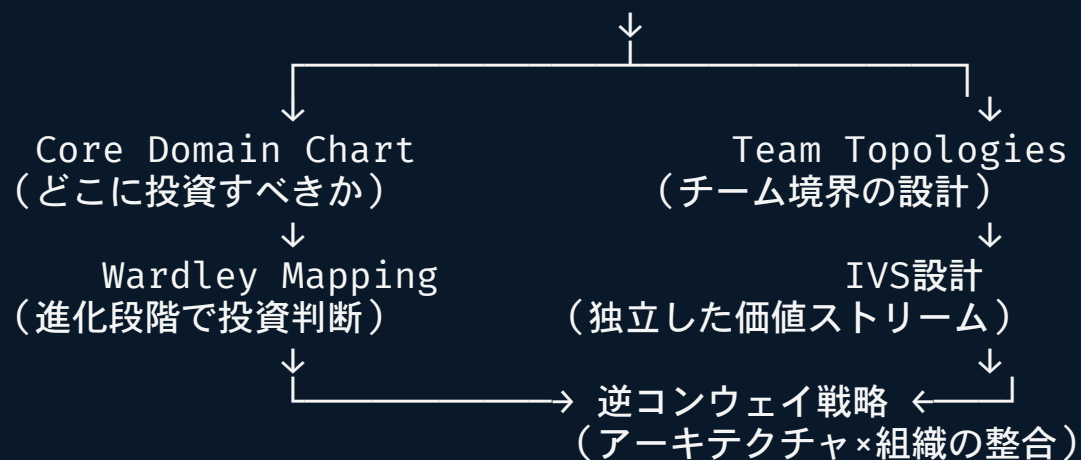
認知負荷を最小化し、チームは自分のドメインに集中できる。コンテキストマップでチーム間関係が明確になり、ドメイン内の変更は同じチーム内で完結する。ビジネスの成長に合わせて**スケーラブルにチームを増やせる**。



概念の関係性マップ

本日紹介した概念はすべて繋がっている

イベントストーリーミング → Bounded Context の発見



イベントストーリーミングで発見し、DDDで分析し、Team Topologiesで組織化する

なぜチーム境界を「小さく」保つのか？

ここまでの話の核心

- ドメイン境界＝チーム境界にすることで、自律性と疎結合を実現する
- でも、チームが担当する範囲が広すぎたら？

問い：1チームでどこまでの範囲を担当できるか？

答えは「認知負荷の限界内」まで。これを超えると、チームは機能しなくなる。

→ Team Topologiesが「認知負荷」を重視する理由を理解する



認知負荷（Cognitive Load）とは

人間の脳が同時に処理できる情報量には限界がある

John Swellerが1988年に提唱した認知負荷理論に基づく概念。Team Topologiesでは、チームが担当する範囲を「認知負荷の限界内」に収めることを重視する。人間のワーキングメモリには限界があり、それを超えるとパフォーマンスが急激に低下する。

"Cognitive load is the primary constraint"

認知負荷こそがチーム設計における最大の制約

認知負荷の3種類

種類	説明	対策
Intrinsic（内在性）	問題固有の複雑さ	削減困難。受け入れる
Extraneous（外因性）	環境・ツールの複雑さ	削減すべき
Germane（関連性）	学習のための負荷	適度に維持



認知負荷を管理する

プラットフォームチームの役割

開発者体験を向上させ、「舗装された道」を提供することで、Stream-alignedチームの認知負荷を軽減する。

チーム設計の原則

1チームが担当する範囲を限定し、依存関係を最小化。
自律的に動ける単位にする。

チームに「何が一番面倒？」と聞け。その答えがExtraneous Loadであり、最初に潰すべき標的だ



持続可能な高速フロー

Team Topologiesの核心目標

速度だけでなく、品質を維持しながら何年も続けられる持続可能なフロー。

高速フローを阻害するもの

組織的な障壁

過剰な承認プロセス、チーム間の依存関係、不明確な責任範囲、サイロ化された知識。**人の問題。**

技術的な障壁

モノリシックなコードベース、共有データベース、手動デプロイプロセス、テスト不足。**技術の問題。**

Accelerate (Forsgren et al., 2018) の研究が示した事実：チームの自律性とデリバリー性能は強く相関する
高速フローは組織の健全性を測る物差しーベロシティ指標ではなく



なぜチームか

問い：優秀な個人を集めれば、優秀なチームになるか？

答えは No。個人のスキルの総和がチームの能力ではない。チームには独自のダイナミクスがあり、それが成果を左右する。

従来のアプローチの問題

リソース思考

「この人をあのプロジェクトに」一々スキルセットだけで編成し、個人の専門性に依存する。**人を部品として扱う発想。**

なぜ問題か

チームの文脈が失われ、暗黙知が継承されず、信頼関係を毎回再構築。**毎回ゼロからスタート**になる。

人を「リソース」と呼ぶ組織は、チームを育てられない。名前と呼べ。



チームファーストの実践

チームを設計の基本単位として扱う

システムを設計するとき、「どのチームが担当するか」を最初に考える。個人ではなく、チームの認知負荷・スキル・成長を考慮する。

具体的なアプローチ

設計時の考慮

チームの認知負荷を超えない範囲に留め、依存関係を最小化し、**自律的に意思決定できる単位**にする。

運用時の考慮

チームの学習と成長を支援し、長寿命チームを維持。**チーム間コラボレーション**も意図的に設計する。

「意志を実装する」との関係

チームが自律的に動けなければ、意志は実装できない。**チームの自律性を確保することが、意志を実装する前提条件。**



疎結合アーキテクチャで自律性を実現する

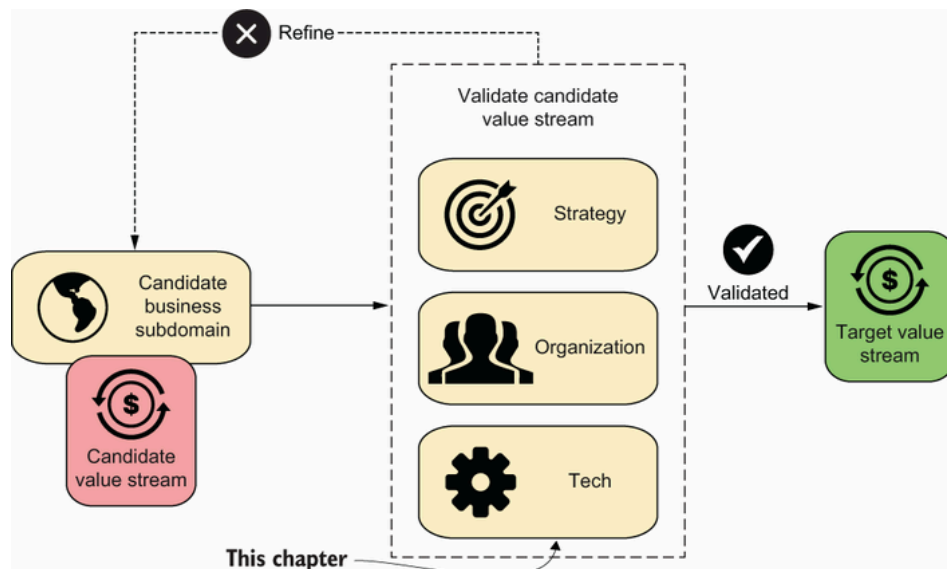


Figure 12.1 Loosely coupled architecture より引用

チームの自律性には、技術的な疎結合が前提

Team Topologiesで定義したチーム境界も、システムが密結合のままでは機能しない。あるチームの変更が別チームのデプロイを止める状況では、自律性は絵に描いた餅。

疎結合の3原則: 独立デプロイ可能、障害の伝搬を防ぐ、チーム間の調整コストを最小化

チームの自律性を支える基盤

ここまでの話

Stream-alignedチームが価値を届けることに集中するには、認知負荷を軽減する必要がある。では、誰がその負荷を引き受けるのか？

答え：プラットフォームチームとIDP (Internal Developer Platform)

各チームがインフラ、CI/CD、監視をゼロから作る必要はない。共通基盤として「舗装された道」を提供すれば、チームはビジネスロジックに集中できる。

→ これがTeam Topologiesにおける**Platform Team**の役割



Internal Developer Platform (IDP)

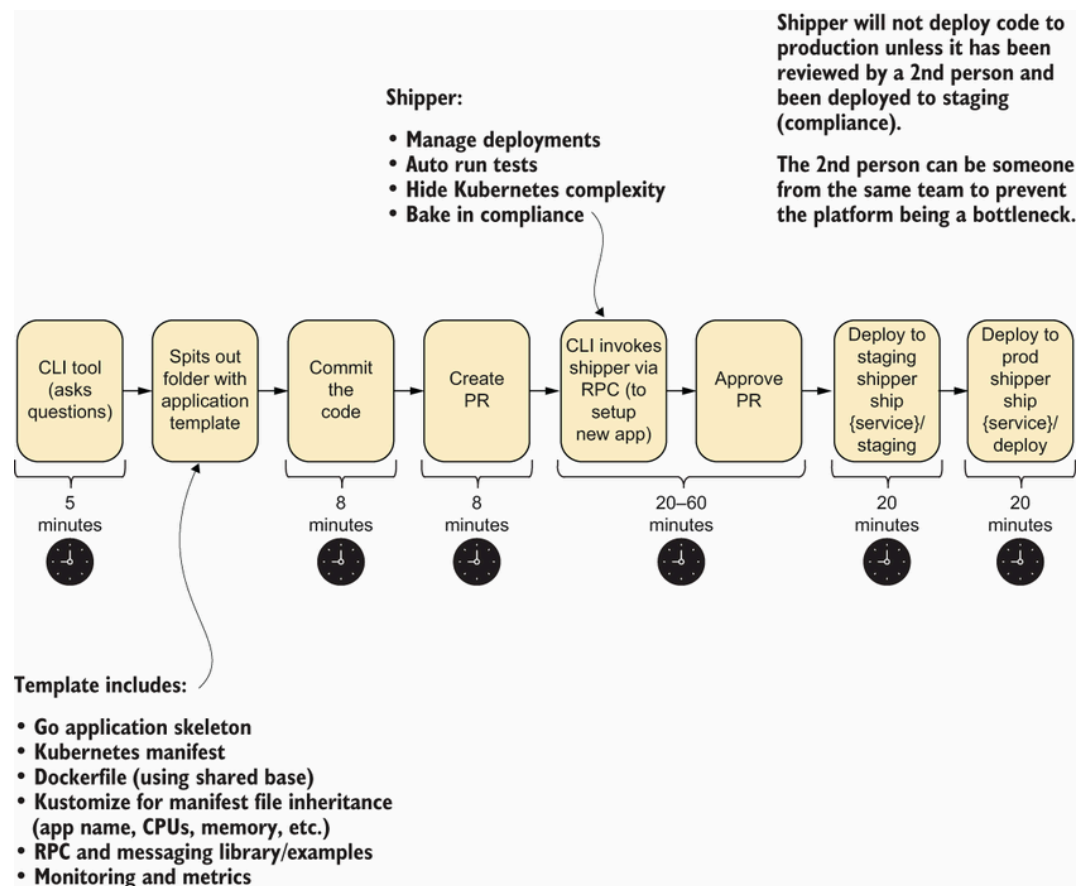


Figure 13.1 Golden paths (paved roads) より引用

問い：各チームがインフラ、CI/CD、監視を個別に作るべきか？

答え：No。共通基盤として「舗装された道 (Paved Road)」を提供する。Netflixが提唱した概念で、強制ではなく「この道を通ると楽」という設計思想。

セルフサービス機能

環境プロビジョニング、CI/CDパイプライン、モニタリング・ログ。必要なものを自分で取得できる。

ゴールデンパス

推奨されるやり方、テンプレート提供、ガードレール。迷わず正解にたどり着ける道。

開発者は「どうやるか」を考えずに「何を作るか」に集中できる。

"Stream-aligned teams can deliver independently when domains are properly decoupled"

ドメインが適切に分離されていれば
Stream-alignedチームは独立してデリバリーできる

現実のレガシーとどう向き合うか

設計手法は揃った。しかし現実のレガシーシステムは理論通りにはいかない。

フェーズ	手法	得られるもの
現状理解	リスニング/マッピングツアー	痛み・願望・政治
可視化	イベントストーリーミング	ビジネスフロー、ホットスポット
境界定義	DDD (Bounded Context)	責任範囲、連携パターン
チーム設計	Team Topologies	自律的なチーム構造
境界の強制	型システム (関数型DDD)	AIが破れない壁

残る問い

道具は揃った。だが目の前には、10年以上動き続けてきたレガシーシステムがある。これを理論通りにきれいに移行できるのか？

→ 段階的・継続的な進化のアプローチへ



"Legacy architecture is a business liability, not merely a technical problem"

レガシーアーキテクチャは単なる技術的問題ではない
ビジネス上の負債である

5. レガシーを 競争優位性へ転換する

レガシーの本当の問題は現場で起きている

「古いから問題」ではない。「変化に対応できないから問題」

レガシーシステム自体は悪ではない。問題は、ビジネスが求める速度で変化できないこと。ソフトウェアの価値は初期構築にではなく、変化への継続的適応にある。だからシステムは「完成」しない——作った瞬間から適応の旅が始まる。

あなたの現場で、こんなことが起きていないか？

開発者が感じる痛み

- 小さな変更の数週間かかる
- 「ここを触ると何が壊れるかわからない」
- テストがないので怖くて触れない
- 新しい技術を試す余裕がない

ビジネスを感じる痛み

- 競合は3日で出す機能が、うちは3ヶ月
- 「技術的に難しい」と言われ続ける
- 見積もりが常に大きく外れる
- 優秀なエンジニアが辞めていく

モグラ叩きをやめろ。構造を変えない限り、問題は永遠に湧き出る。

「構造を変える余裕がない」——それは構造を変えないことのコストを見えていないだけ



レガシーの本当の問題は負のサイクルにある

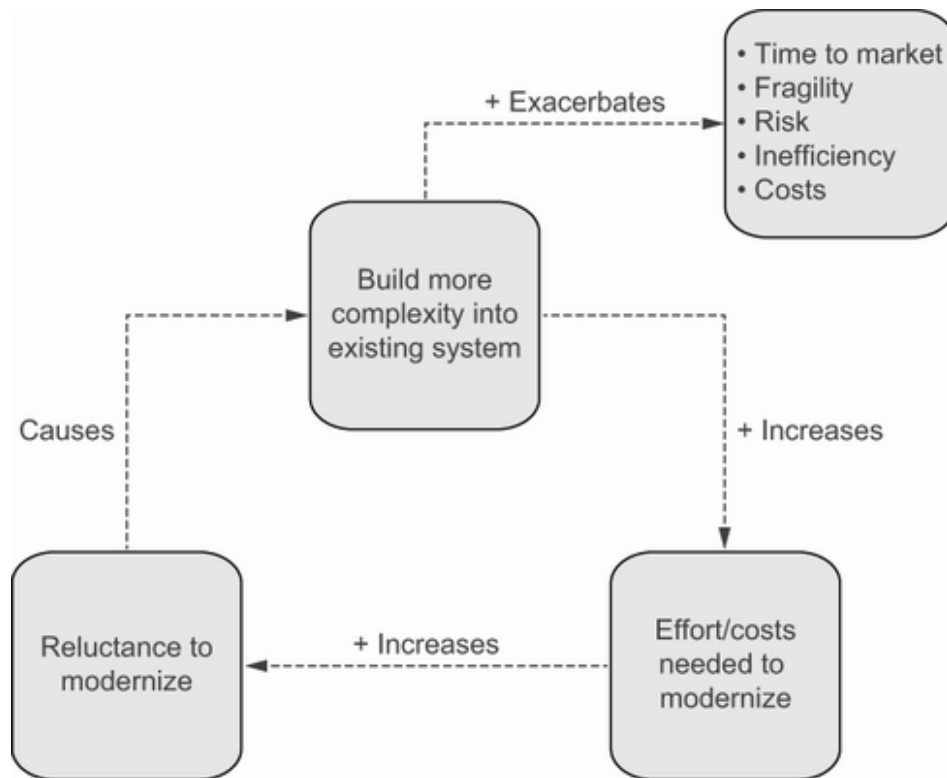


Figure 1.1 The negative cycle of declining architecture health より引用

なぜ問題は悪化し続けるのか？

1. 変更が困難 → デリバリーが遅れる
2. デリバリーが遅い → ビジネスからの信頼低下
3. 信頼低下 → 投資が減少（「どうせ遅いから」）
4. 投資減少 → 技術的負債がさらに蓄積
5. 負債の蓄積 → さらに変更が困難に（最初に戻る）

このサイクルは**自己強化**する。放置すればするほど、抜け出すのが難しくなる。

サイクルを断ち切るには、どこかに「くさび」を打つ必要がある

負のサイクルを断ち切る「くさび」

5つのポイントのどこかに介入する

サイクル全体を一度に変えることはできない。1箇所に集中して「くさび」を打つ。

介入ポイントと戦略

介入点	具体的な戦略	難易度
変更の困難さ	モジュール化、テスト追加	高（時間がかかる）
デリバリー速度	CI/CD、小さなリリース	中（ツールで支援可能）
信頼低下	可視化、小さな成功の積み重ね	中（コミュニケーション）
投資減少	ビジネス価値の可視化	高（経営層の理解が必要）
負債の蓄積	技術負債の定量化、優先順位付け	中

最初の1勝が、組織を動かす。小さく勝て。



小さな成功が次を呼ぶ

負のサイクルを「正のサイクル」に転換する

[負のサイクル]

変更困難 → 遅延 → 不信

↑

↓

負債蓄積 ← 投資減少

[正のサイクル]

小さな成功 → 信頼回復 → 投資増加

↑

↓

能力向上 ←—— 改善加速

最初の「小さな成功」を選ぶ基準

1. ビジネスに見える成果: 技術者だけが喜ぶ改善は避ける
2. 3ヶ月以内に達成可能: 長すぎると信頼を失う
3. 再現可能な方法: 一度きりの英雄的行為ではなく、プロセスとして

最初の成功は「証明」のため。2回目以降は「拡大」のため。



AMET: モダナイゼーション実行フレームワーク

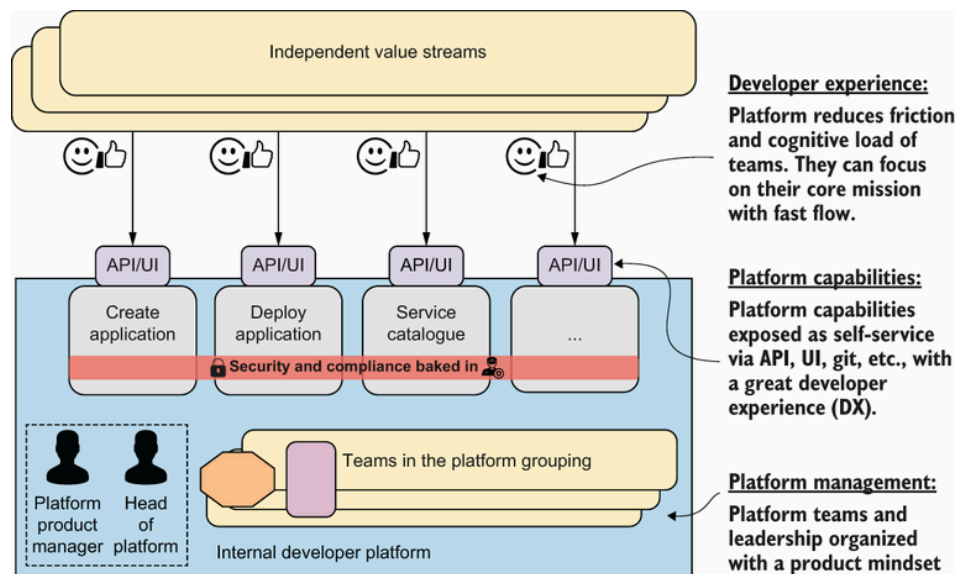


Figure 15.1 AMET: Architecture Modernization Enabling Team より引用

AMET = Architecture Modernization Enabling Team

モダナイゼーションを「誰かの片手間」にしない。専任のイネーブリングチームが、組織全体の変革を推進する。

AMETの役割: 技術的な移行計画の策定、チームへのコーチング、組織横断的な障害の除去。Team TopologiesにおけるEnabling Teamとして機能する。

段階的な進化のアプローチ

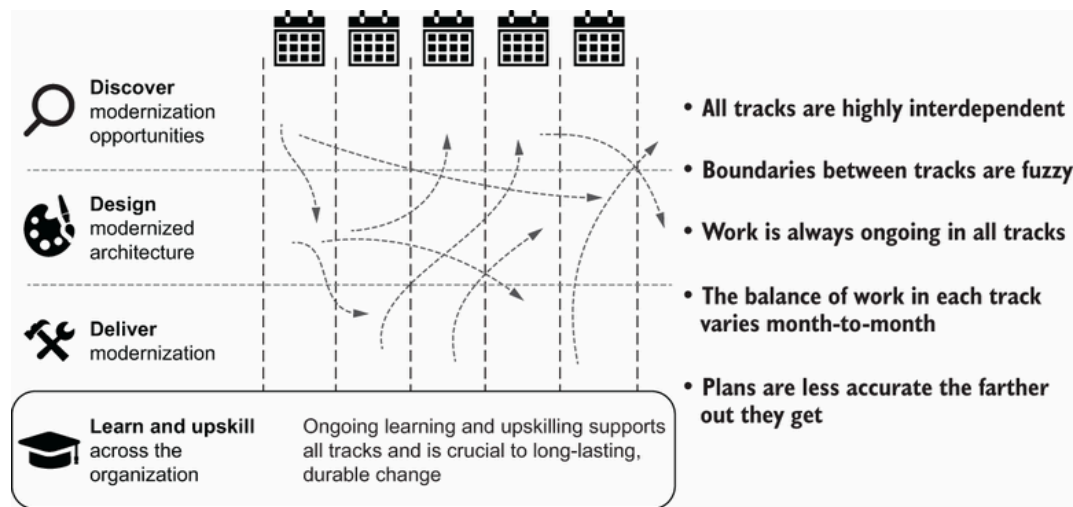


Figure 16.2 Parallel streams of work より引用

モダナイゼーションを「プロジェクト」と呼んだ瞬間、失敗が始まる。プロジェクトには終わりがある。だが組織の進化に終わりはない。

Portfolio-Driven Evolution

- 発見 → 設計 → デリバリー → 学習
- これらが**並行して継続的に**進む
- ビッグバンリプレイスは失敗する

早期の価値提供

- 小さな改善を積み重ね、継続的に価値を届ける
- 学習をフィードバックして計画を調整

並行ストリームの実践

4つのストリームを同時に回す

発見・設計・デリバリー・学習ーこれらは順番にやるものではなく、常に並行して走らせる。

各ストリームの活動

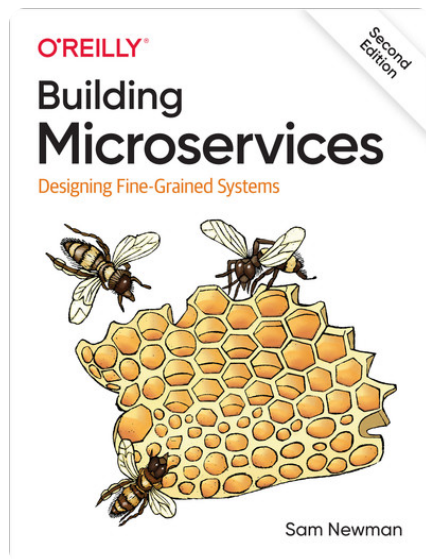
ストリーム	活動	成果物
Discovery	リスニングツアー、マッピング	現状の可視化、問題の特定
Design	イベントストーリーミング、境界設計	目標アーキテクチャ
Delivery	段階的な移行、機能開発	動くソフトウェア
Learning	振り返り、メトリクス分析	次のイテレーションへの入力

「設計が終わってから実装」ではない。学びながら設計し、設計しながら実装する。



Newmanが語る「進化的アーキテクト」

Sam Newmanは **Building Microservices** の最終章（Ch.16 "The Evolutionary Architect"）で、アーキテクトの役割を再定義している。



Sam Newman, O'Reilly, 2021

アーキテクトは「完璧な設計図を描く人」ではなく「変化を可能にする構造を設計する人」

Newmanの主張は明快。**独立デプロイ可能性**こそがマイクロサービスの唯一の本質的価値であり、それが不要ならモノリスでいい。情報隠蔽と低結合を徹底し、「知りすぎない」境界を引くことが設計者の最も重要な仕事。そして最大のスケーリングコストは技術ではなく**チーム間の調整コスト**。

これは本書Architecture Modernizationの「意志を実装する」と同じ構図。Newman自身もConway's Lawを取り上げ、**組織がアーキテクチャを形作る**ことを前提にチーム設計を語っている。

「進化的アーキテクト」とは「意志を持って構造を選ぶ人」のこと。

完璧な設計は存在しない。変化に耐える設計だけが生き残る。それを選ぶのがアーキテクトの意志。

「作らない」の先に何を作るか

車輪の再発明をやめてリソースが浮いた。そのリソースをどこに向けるか？

「現状維持でよい」は選択肢にない。効率化には理論的上限があり、効率化だけを価値とするプロダクトは、より安い代替に置き換えられる運命にある。次の問いは「何に進化させるか」。

→ 記録するだけの仕組みを、洞察を生む仕組みに進化させること。それがモダナイゼーションの真の目的地。

記録のシステムから洞察のシステムへ

「代替されない」ことは「現状維持でよい」ことを意味しない

効率化には理論的上限がある。入力を受けて記録するだけの System of Record（記録のシステム）は、いずれコモディティ化する。効率化だけを価値とするプロダクトは、より安いプロダクトに置き換えられる運命にある。

System of Record

データを正確に記録・管理する。ERPや基幹システムの多くがここに位置する。**必要不可欠だが、差別化にはならない。**

System of Insight

蓄積されたデータから未来を予測し、意思決定を支援する。「何が起きたか」ではなく「**次に何をすべきか**」を提示する。ここに事業の持続的成長がある。

モダナイゼーションの真の目的地は、単にレガシーを新しくすることではない。記録するだけのシステムを、**洞察を生むシステムに進化**させること。それは技術の入れ替えでは達成できず、ドメイン理解の深さによって決まる。

事業の持続的成長は技術要素ではなく、提供価値の深さによって決まる



なぜ「意志」が必要なのか

モダナイゼーションは「正解を実行する」作業ではない

最適なアーキテクチャ、最適なチーム構造、最適な技術選定——そんなものは存在しない。あるのは「トレードオフ」と「制約」と「不確実性」だけ。

誰かに決めてもらうことの危険

ベンダーの「ベストプラクティス」、コンサルタントの「業界標準」、本に書いてある「推奨アプローチ」——**自社の文脈を無視した解は機能しない。**

意志を持つとは

自社の強み・弱みを理解した上で選び、トレードオフを引き受ける覚悟を持ち、選択の結果に責任を持つ。**文脈を理解した者だけが正しく選べる。**そして「今のままで十分」は意志ではない。現状維持は緩やかな後退に過ぎない。

「今のままでいい」は意志ではない。緩やかな後退に過ぎない。



失敗パターンと成功パターン

失敗パターン

技術だけに注力し組織を無視、ビッグバンリプレイス、現場を巻き込まないトップダウン、銀の弾丸への期待、短期的な成果を求めすぎる、学習投資の軽視。

成功パターン

ソシオテクニカルなアプローチ、段階的・継続的な改善、協働的な設計プロセス、ビジネス成果との明確な紐づけ、長期的なコミットメント、学習文化の醸成。

"If leaders are still looking for quick fixes, they don't understand the problem"

リーダーがまだ銀の弾丸を探しているなら、問題を理解していない

"Modularization is not just about code—it's about enabling business agility"

モジュール化は単なるコードの話ではない
ビジネスのアジリティを実現することだ

旅を振り返る

たどってきた道 — 「意志を実装する」ための5つのステップ

ステップ	内容	意志との関係
1. AI時代	AIが「How」を加速、人間は「What」「Why」へ	意志の必要性を理解
2. ソシオテクニカル	技術と組織を同時に設計	意志の対象を理解
3. 協働的手法	イベントストーミングでドメインを発見	意志を言語化する
4. DDD × TT	境界を揃えて自律的チームへ	意志を構造化する
5. レガシー脱却	段階的・継続的に進化	意志を実行する

道具は揃っている。足りないのは「この組織をどうしたいか」という意志だけ。

6. 意志を実装すること

翻訳者として伝えたいこと

この本を訳しながら、私はずっと考えていました。なぜ同じ失敗が繰り返されるのか。なぜ「正しいこと」を知っている人がたくさんいるのに、組織は変わらないのか。

私の結論はこうです。モダナイゼーションの本質は、完成形を目指すことではない。変化し続ける能力を組織に埋め込むこと。

技術を一新しても、変化に対応し続ける文化がなければ、またレガシーに戻る。逆に、変化し続ける構造があれば、技術は後からついてくる。問題は技術ではなく、組織が変化を受け入れる仕組みを持っているかどうか。

本日のまとめ——意志を実装するとは

AIがHowのコストを下げた結果、ボトルネックは人間の調整に移った。モダナイゼーションは「負債を返す」作業ではない。変化し続ける能力を組織に埋め込む作業。冒頭で定義した「3つの意志」に何が見えたか。

ビジネスへの意志

Core Domainに集中し、Generic Subdomainは外部に任せる。

→ イベントストーリーミング、Core Domain Chart

アーキテクチャへの意志

Bounded Contextで境界を定義し、コンテキストマッピングで関係を設計する。

→ DDD、Strangler Fig、IDP

組織への意志

逆コンウェイ戦略でドメイン境界＝チーム境界にし、認知負荷の限界内で設計する。

→ Team Topologies、IVS

持ち帰るべきは手法ではない。「自分の組織をどうしたいか」という問いだ。



AI時代に「意志」はどう変わるか

「AIがいずれWhatやWhyも判断するのでは？」——最もよく受ける質問に答えたい。

仮にそうなったとしても、その判断を**採用する責任**は人間にしかない。AIが「このドメインは外部に委託すべき」と提案しても、その結果として起きるレイオフの責任をAIは取らない。意志とは判断することではなく、**判断の結果を引き受ける覚悟**のこと。

AIエージェントがいる世界の落とし穴

「作れるから作る」が加速する。コストが低いから、考えずに試す。結果、**方向性なき大量生産**に陥る。技術的負債も倍速で溜まる。

だからこそ「意志」が要る

何を作らないかを決める力、どこに境界を引くかの判断、組織の方向性を定める覚悟。**AIが速く走れるからこそ、人間がハンドルを握る意味がある。**

速く走れるマシンほど、操縦者の判断が生死を分ける



今日から始められること

来週やること（30分で終わる）

チームの定例で「最も変更が怖いコード」を1つ挙げる。怖い理由を付箋に書き出す。技術の問題（テストがない、依存が複雑）か、組織の問題（担当者が不明、承認が必要）かを仕分ける。**仕分け結果がモダナイゼーションの出発点になる。**

来月やること（意思決定の棚卸し）

「誰かが決めてくれるのを待っている」案件を1つ特定する。「なぜ決まらないのか」を3つの観点で書き出す——権限がない、情報がない、合意できない。そのうち**自分で動かせるものから着手する**。全部は無理でも、1つ動かすだけで景色が変わる。



本日、夕方からイベントがあります

3-shake SRE Tech Talk 特別回

アーキテクチャモダナイゼーション

2026.2.19(木)19:30~21:00(19:00開場)

会場：東京都中央区銀座8丁目21番1号
住友不動産汐留浜離宮ビル7F

ハイブリッド開催

アーキテクチャモダナイゼーション
組織とビジネスの未来を設計する

何度書き直しても、
また遅くなる。

◆ その実装部は正確に、コードではなく、組織構造にある。
◆ 最新のモダナイズメーションの
ユフイロの仕組みから理解される組織はない。
◆ DDD、チームトポロジー、フォードリーディング
— 世界の現場が採用した最先端

3-shake SRE Tech Talk 特別回 アーキテクチャモダナイゼーション

レガシーシステムを競争優位性に転換するための包括的ガイド。ソシオテクニカルな視点、イベントストーリーミング、DDD、Team Topologiesなど、本セッションで紹介した内容をより深く学べます。

<https://3-shake.connpass.com/event/382086/>

参考資料

- [Architecture Modernization](#) - Nick Tune, Jean-Georges Perrin (Manning, 2024)
- [アーキテクチャモダナイゼーション](#) - 日本語版、株式会社スリーシェイク訳
- [Team Topologies](#) - Matthew Skelton, Manuel Pais (IT Revolution, 2019)
- [チームの力で組織を動かす](#) - 松本成幸 (技術評論社, 2025)
- [企業変革のジレンマ](#) - 宇田川元一 (日経BP, 2024)
- [Building Microservices, 2nd Edition](#) - Sam Newman (O'Reilly, 2021)
- [Domain-Driven Design](#) - Eric Evans (Addison-Wesley, 2003)
- [関数型ドメインモデリング](#) - Scott Wlaschin (KADOKAWA, 2025)
- [EventStorming](#) - Alberto Brandolini
- [Sooner Safer Happier](#) - Jon Smart (IT Revolution, 2020)
- [現代システムの三体問題](#) - Architecture Modernization 読書感想文
- [関数型ドメインモデリング](#) 読書感想文
- [信頼と不信の哲学入門](#) - キャサリン・ホーリー (岩波新書, 2024)



ありがとうございました

ご質問・ご相談はお気軽にお問い合わせください

@nwiizo | <https://3-shake.com>